

Portal CJR — Documentação Completa

Sistema: Portal CJR — Plataforma de Gestão de Manutenção Preventiva

Empresa: Costa Júnior (CJR)

Domínio: portalcjr.vip

Data: Junho de 2026

Índice de Documentos

#	Documento	Descrição
01	Visão Geral do Sistema	Objetivo, público-alvo, funcionalidades e fluxos principais
02	Arquitetura Completa	Stack tecnológica, estrutura de pastas, autenticação e S3
03	Modelagem de Dados	Todas as tabelas com campos, tipos e relacionamentos
04	Documentação das Telas	Mapa de rotas, campos, botões e navegação de cada tela
05	Documentação das APIs	Todas as procedures tRPC com inputs, outputs e exemplos
06	Regras de Negócio	Precificação, cronograma, fluxos de aprovação e cálculos
07	Usuários e Permissões	Perfis, permissões por módulo e verificação de acesso
08	Manual de Implantação	Instalação, configuração, variáveis de ambiente e deploy
09	Manual do Usuário	Guias passo a passo para cliente, técnico e administrador
10	Código Fonte Organizado	Estrutura de pastas, padrões de código e arquivos críticos
11	Roadmap Futuro	Bugs conhecidos, melhorias planejadas e débitos técnicos

Diagramas

Diagrama	Formato	Descrição
Arquitetura do Sistema	PNG	Visão geral da arquitetura (browser → servidor → banco → serviços)
Banco de Dados (ER)	PNG	Diagrama entidade-relacionamento do módulo de manutenção
Fluxo de Visita Preventiva	PNG	Fluxo completo de execução de uma visita preventiva
Fluxo de Negociação de Orçamento	PNG	Estados e transições do fluxo de orçamentos
Fluxo de Autenticação	PNG	Sequência de autenticação OAuth e e-mail+senha

Exportações

Arquivo	Formato	Conteúdo
Portal_CJR_Documentacao_Completa.pdf	PDF	Todos os 11 documentos em um único PDF
Portal_CJR_Documentacao_Completa.docx	Word	Todos os 11 documentos em formato editável

Resumo do Sistema

O Portal CJR é uma plataforma web completa para gestão de manutenção preventiva, construída com:

- **Frontend:** React 19 + TypeScript + Tailwind CSS 4 + shadcn/ui
- **Backend:** Node.js + Express 4 + tRPC 11 + Drizzle ORM
- **Banco:** TiDB (compatível MySQL)
- **Arquivos:** AWS S3
- **Autenticação:** Manus OAuth (internos) + E-mail/Senha (clientes e técnicos)
- **Pagamentos:** Mercado Pago

O sistema atende três perfis: **Administrador CJR** (gestão completa), **Técnico de Manutenção** (execução de visitas com checklist e PDF) e **Cliente/Lojista** (acompanhamento, aprovações e orçamentos).

Portal CJR — Documentação Completa

Documento 01: Visão Geral do Sistema

Empresa: Costa Júnior (CJR)
Sistema: Portal CJR — Plataforma de Gestão de Manutenção Preventiva
Domínio: portalcjr.vip
Versão documentada: Checkpoint 8317a97c
Data de referência: Junho de 2026

1.1 Objetivo do Aplicativo

O Portal CJR é uma plataforma web completa de gestão de manutenção preventiva desenvolvida para a empresa Costa Júnior (CJR). O sistema centraliza todo o ciclo de vida dos contratos de manutenção: desde a captação de novos clientes (lojistas de shopping centers) até a execução das visitas técnicas preventivas, geração de relatórios em PDF com assinatura digital, controle financeiro e negociação de orçamentos de serviços extras.

O sistema resolve a necessidade de digitalizar e automatizar um processo que anteriormente era conduzido de forma manual — agendamentos em planilhas, relatórios em papel e comunicações por WhatsApp —, oferecendo rastreabilidade completa, histórico de visitas, evidências fotográficas e fluxos de aprovação estruturados.

1.2 Público-Alvo

O sistema atende três perfis distintos de usuário, cada qual com seu próprio portal de acesso:

Perfil	Portal de Acesso	Descrição
Técnico de Manutenção	/tecnico	Profissional de campo que executa as visitas preventivas e corretivas nas lojas dos clientes.
Cliente (Lojista)	/cliente	Responsável pela loja contratante do plano de manutenção preventiva. Acompanha visitas, aprova materiais e orçamentos.
Administrador CJR	/admin/manutencao e /manutencao/*	Equipe interna da CJR que gerencia clientes, técnicos, agendamentos, orçamentos e financeiro.

Além desses três perfis principais do módulo de manutenção, a plataforma também serve colaboradores internos da CJR (perfis: admin, coordenador, administrativo, financeiro, operacional, comercial) que utilizam outros módulos do sistema (Fórum de Dúvidas, Treinamentos, Gestão Comercial, etc.).

1.3 Principais Funcionalidades

Módulo de Manutenção Preventiva (foco desta documentação)

Para o Administrador CJR:

- Cadastro e gestão completa de clientes (lojistas) com código único (ex.: CLT-0001)
- Gestão de técnicos de manutenção com vinculação a clientes específicos
- Simulador de preços com cálculo automático por tipo de loja, serviços e plano
- Fluxo de pré-cadastro (leads) com Kanban de prospecção e integração Mercado Pago
- Geração automática de cronograma de preventivas (1 visita/mês conforme plano)
- Painel de detalhe do cliente com 6 abas: Lojas, Preventivas, Relatórios, Materiais, Financeiro e Orçamentos
- Upload e envio de orçamentos em PDF para clientes
- Aprovação de solicitações de material com taxa administrativa (~30%)
- Gestão de chamados de suporte abertos por técnicos
- Configuração de texto do contrato de adesão
- Exportação de dados em CSV

Para o Técnico:

- Visualização de lojas vinculadas e agenda de visitas
- Checklist de preventiva com 36 itens (Civil, Hidráulica, Elétrica) com foto obrigatória por item
- Abertura e execução de chamados corretivos com foto do problema
- Solicitação de materiais durante a visita
- Assinatura digital do cliente ao final da visita
- Geração automática de PDF do relatório com logo CJR, dados da visita, fotos e assinatura
- Aba de suporte técnico para abertura de chamados internos

Para o Cliente:

- Dashboard com gráfico de chamados por tipo (anual)
- Visualização de todas as lojas do mesmo grupo (mesmo clientCode)
- Acompanhamento de chamados com foto e localização do problema
- Histórico de visitas preventivas com acesso ao PDF do relatório
- Aprovação ou rejeição de solicitações de material com upload de comprovante de pagamento
- Fluxo de negociação de orçamentos: aprovação, rejeição ou sugestão de ajuste (ciclo iterativo)
- Acesso ao contrato de adesão
- Edição de perfil (e-mail e telefone)

Outros Módulos da Plataforma

- **Fórum de Dúvidas:** Chat com IA (LLM) alimentado por base de conhecimento, com escalção para resposta humana
- **Treinamentos:** Biblioteca de vídeos e PDFs organizados por área (Administrativo, Financeiro, Operacional, Comercial)
- **Gestão Comercial:** CRM de prospecção com Kanban, histórico de interações, tarefas e templates de e-mail
- **Documentos:** Geração de Termos de Entrega e Atas de Reunião com assinatura digital
- **Gestão Técnica:** Controle de estoque (inventário) e equipamentos

1.4 Fluxo Geral de Utilização

Fluxo de Contratação (Novo Cliente)

1. Lojista acessa /manutencao/contratar
2. Preenche dados da loja e simula o preço
3. Escolhe o plano (trimestral/semestral/anual)
4. Assina o contrato digitalmente
5. Realiza o pagamento via Mercado Pago
6. Lead é registrado no sistema (status: "novo")
7. Admin CJR converte o lead em cliente ativo
8. Admin gera link de criação de senha para o cliente
9. Admin gera cronograma automático de preventivas
10. Cliente recebe acesso ao portal /cliente

Fluxo de Visita Preventiva

1. Admin agenda visita para um técnico
2. Técnico acessa /tecnico e vê a agenda
3. Técnico executa o checklist de 36 itens com fotos
4. Técnico coleta assinatura digital do responsável da loja
5. Sistema gera PDF do relatório automaticamente
6. PDF fica disponível para o cliente em /cliente (aba Preventivas)
7. Admin visualiza o relatório na aba Relatórios do detalhe do cliente

Fluxo de Solicitação de Material

1. Técnico solicita material durante a visita (descrição + valor estimado)
2. Cliente recebe notificação e aprova ou rejeita em /cliente
3. Se aprovado, cliente realiza o pagamento
4. Cliente faz upload do comprovante de pagamento
5. Admin visualiza o comprovante na aba Materiais

Fluxo de Orçamento de Serviço Extra

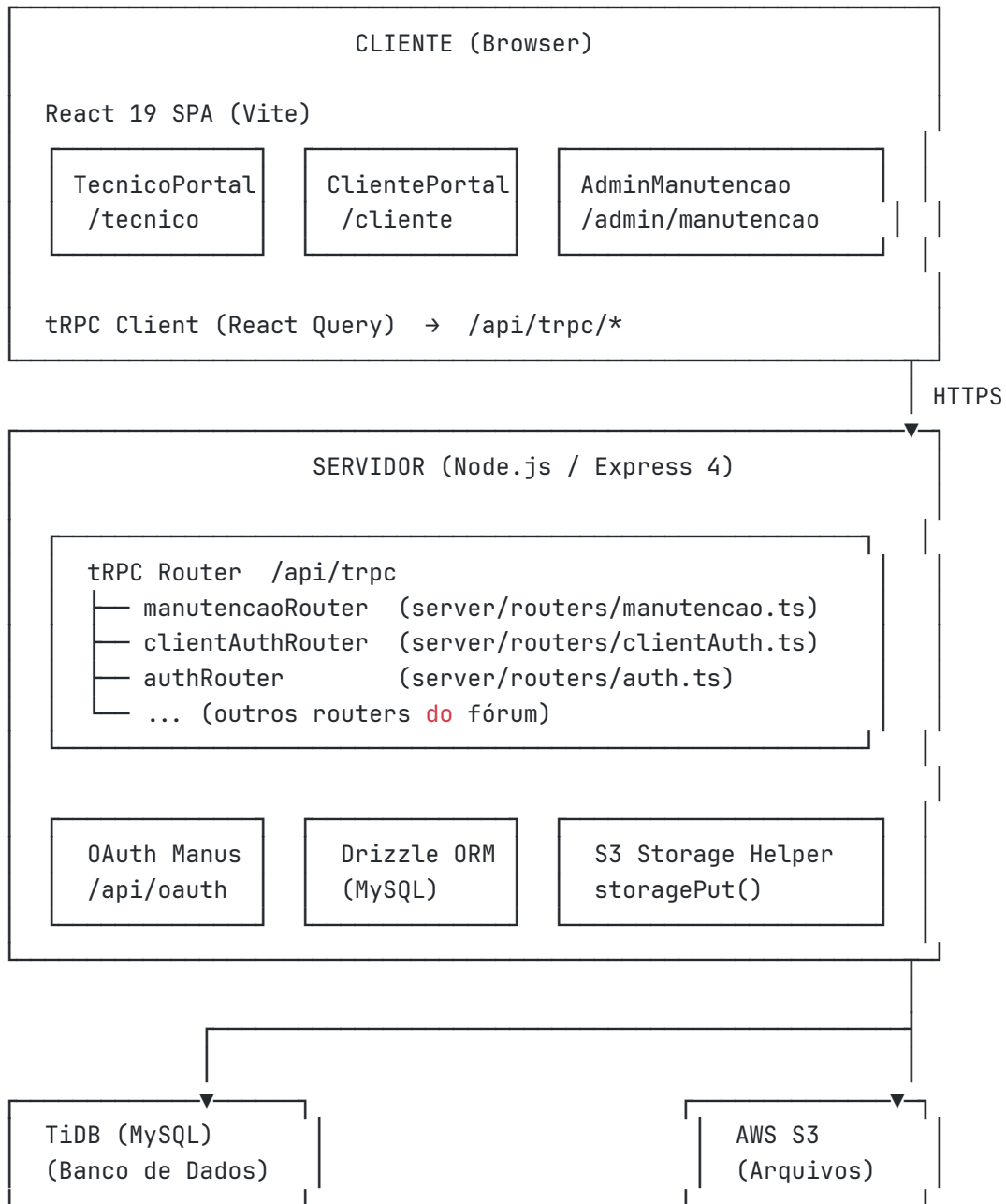
1. Admin cria orçamento e faz upload do PDF
2. Cliente recebe o orçamento em /cliente (aba Orçamentos)
3. Cliente pode: Aprovar | Rejeitar | Sugerir Ajuste
4. Se "Sugerir Ajuste": cliente descreve a sugestão
5. Admin vê a sugestão e pode reenviar novo PDF (nova rodada)
6. Ciclo se repete até aprovação ou rejeição definitiva

Portal CJR — Documentação Completa

Documento 02: Arquitetura Completa

2.1 Estrutura do Sistema

O Portal CJR é uma aplicação web monolítica com separação clara entre frontend (SPA React) e backend (API Express/tRPC). Ambos rodam no mesmo processo Node.js em desenvolvimento, mas são servidos de forma independente em produção via Vite (frontend estático) e Express (API).



2.2 Tecnologias Utilizadas

Frontend

Tecnologia	Versão	Finalidade
React	19	Framework de UI
TypeScript	5.x	Tipagem estática
Vite	6.x	Bundler e dev server
Tailwind CSS	4.x	Estilização utilitária
shadcn/ui	—	Componentes de UI (Radix UI)
Wouter	—	Roteamento SPA (alternativa ao React Router)
@tanstack/react-query	5.x	Cache e estado de servidor
@trpc/react-query	11.x	Cliente tRPC com React Query
jspdf	—	Geração de PDF no browser (relatório de preventiva)
signature_pad	—	Assinatura digital no canvas
recharts	—	Gráficos (dashboard do cliente)
lucide-react	—	Ícones
framer-motion	—	Animações
date-fns	—	Manipulação de datas
react-hook-form	—	Formulários
zod	—	Validação de schemas
sonner	—	Notificações toast

Backend

Tecnologia	Versão	Finalidade
Node.js	22.x	Runtime
Express	4.x	Servidor HTTP
tRPC	11.x	API type-safe (RPC sobre HTTP)
Drizzle ORM	—	ORM para MySQL/TiDB
mysql2	—	Driver MySQL
bcryptjs	—	Hash de senhas
jose	—	JWT (sessões OAuth)
cookie-parser	—	Parse de cookies
superjson	—	Serialização avançada (Date, Map, etc.)
nodemailer / resend	—	Envio de e-mails
pdf-lib / pdfkit	—	Manipulação de PDFs no servidor
zod	—	Validação de inputs tRPC
tsx	—	Execução TypeScript sem compilação

Banco de Dados

Tecnologia	Finalidade
TiDB (compatível MySQL)	Banco de dados relacional principal
Drizzle Kit	Migrations e push de schema

Serviços Externos

Serviço	Finalidade
Manus OAuth	Autenticação SSO para usuários internos (colaboradores CJR)
AWS S3	Armazenamento de arquivos (PDFs, fotos, comprovantes, assinaturas)
Mercado Pago	Pagamentos online (planos de manutenção e materiais)
Manus LLM API	IA para o Fórum de Dúvidas
Manus Notification API	Notificações push para o dono do sistema

2.3 Estrutura de Pastas do Projeto

```
forum-cjr/
├── client/                                # Frontend React
│   ├── index.html                        # Entry point HTML
│   ├── public/                          # Assets estáticos (servidos em /)
│   └── src/
│       ├── App.tsx                      # Roteamento principal (Wouter)
│       ├── main.tsx                    # Providers (ThemeProvider, trRPC, QueryClient)
│       ├── index.css                   # Variáveis CSS globais (Tailwind 4)
│       ├── const.ts                   # Constantes (getLoginUrl, etc.)
│       ├── components/
│       │   ├── ui/                    # Componentes shadcn/ui (Button, Dialog, etc.)
│       │   ├── DashboardLayout.tsx    # Layout sidebar para painéis internos
│       │   ├── ManutencaoLayout.tsx   # Sidebar do módulo de manutenção
│       │   ├── ComercialLayout.tsx    # Sidebar do módulo comercial
│       │   └── AIChatBox.tsx          # Componente de chat com IA
│       ├── contexts/
│       │   └── ThemeContext.tsx       # Contexto de tema (light/dark)
│       ├── hooks/
│       │   ├── useMobile.tsx          # Detecção de dispositivo móvel
│       │   └── usePersistFn.ts        # Hook para funções persistentes
│       ├── lib/
│       │   ├── trpc.ts                # Configuração do cliente trRPC
│       │   ├── utils.ts               # Utilitários (cn, formatters)
│       │   └── cjrLogoB64.ts          # Logo CJR em base64 (para PDF)
│       └── pages/
│           ├── Home.tsx               # Landing page / hub de navegação
│           ├── Chat.tsx               # Fórum de dúvidas com IA
│           ├── TecnicoPortal.tsx      # Portal do técnico (/tecnico)
│           ├── ClientePortal.tsx      # Portal do cliente (/cliente)
│           ├── ClienteLogin.tsx       # Login do cliente (/login-cliente)
│           ├── TecnicoLogin.tsx       # Login do técnico (/login-tecnico)
│           ├── ClienteDefinirSenha.tsx # Definir senha (token)
│           ├── TecnicoDefinirSenha.tsx # Definir senha (token)
│           ├── ClienteTrocarSenha.tsx  # Trocar senha (autenticado)
│           ├── TecnicoTrocarSenha.tsx  # Trocar senha (autenticado)
│           ├── admin/
│           │   ├── AdminManutencao.tsx # Painel admin (/admin/manutencao)
│           │   ├── AdminClienteDetalhe.tsx # Detalhe do cliente (/admin/manutencao/
│           │   ├── AdminDashboard.tsx   # Dashboard admin geral
│           │   └── ...                  # Outros painéis admin
│           └── manutencao/
│               ├── DashboardManutencao.tsx # Dashboard gestão (/manutencao/dashboar
│               ├── ClientesManutencao.tsx  # Listagem clientes (/manutencao/cliente
│               ├── ClienteDetalhe.tsx      # Detalhe cliente (/manutencao/clientes/
│               ├── ChamadosManutencao.tsx  # Chamados (/manutencao/chamados)
│               ├── AgendaManutencao.tsx    # Agenda (/manutencao/agenda)
│               └── TecnicosManutencao.tsx  # Técnicos (/manutencao/tecnicos)
```

— PreventivosManutencao.tsx	# Preventivos (/manutencao/preventivos)
— OrcamentosManutencao.tsx	# Orçamentos (/manutencao/orcamentos)
— ContratarManutencao.tsx	# Contratação pública (/manutencao/contratar)
server/	# Backend Node.js/Express
— _core/	# Framework (NÃO editar)
— index.ts	# Entry point do servidor
— trpc.ts	# Configuração tRPC (router, procedures)
— context.ts	# Contexto tRPC (req, res, user)
— oauth.ts	# Manus OAuth handler
— cookies.ts	# Configuração de cookies
— llm.ts	# Helper para LLM (Manus AI)
— notification.ts	# Helper para notificações
— env.ts	# Variáveis de ambiente
— imageGeneration.ts	# Helper para geração de imagens
— routers/	
— manutencao.ts	# Router principal de manutenção (admin)
— clientAuth.ts	# Router de auth cliente/técnico + portais
— _clientDetail.ts	# Procedure adminGetClientDetail (helper)
— auth.ts	# Router de autenticação OAuth
— ...	# Outros routers (fórum, comercial, etc.)
— db.ts	# Helpers de query do banco
— storage.ts	# Helper S3 (storagePut, storageGet)
— routers.ts	# Agregador de todos os routers
drizzle/	
— schema.ts	# Definição de todas as tabelas
— migrations/	# Arquivos de migração gerados
shared/	# Código compartilhado frontend/backend
— types.ts	# Tipos compartilhados
package.json	# Dependências e scripts
tsconfig.json	# Configuração TypeScript
vite.config.ts	# Configuração Vite
drizzle.config.ts	# Configuração Drizzle Kit

2.4 Fluxo de Autenticação

O sistema possui **dois sistemas de autenticação independentes**:

Sistema 1: Manus OAuth (Colaboradores Internos)

- Utilizado por: usuários internos da CJR (admin, coordenador, financeiro, etc.)
- Fluxo: redirecionamento para portal Manus OAuth → callback em `/api/oauth/callback` → cookie de sessão JWT
- Implementado em: `server/_core/oauth.ts`

- Cookie: gerenciado pelo framework `_core`

Sistema 2: E-mail + Senha (Clientes e Técnicos)

- Utilizado por: clientes (lojistas) e técnicos de manutenção
- Fluxo: POST para `clientAuth.clientLogin` ou `clientAuth.techLogin` → bcrypt hash comparison → cookie de sessão próprio
- Cookies: `cjr_client_session` (clientes) e `cjr_tech_session` (técnicos)
- Duração da sessão: 30 dias
- Token: 48 bytes aleatórios (hex), armazenado em `clientSessions` ou `technicianSessions`
- Implementado em: `server/routers/clientAuth.ts`

Redefinição de Senha

- Admin gera token de reset (válido por 24h) e envia link por e-mail
- Cliente/técnico acessa `/cliente/definir-senha?token=XXX` ou `/tecnico/definir-senha?token=XXX`
- Senha padrão para técnicos: `tecnico1234` (deve ser trocada no primeiro acesso)

2.5 Comunicação Frontend ↔ Backend (tRPC)

Todas as chamadas de dados utilizam tRPC sobre HTTP. Não há endpoints REST manuais para funcionalidades de negócio.

```
// Exemplo de query no frontend
const { data } = trpc.manutencao.listClients.useQuery({ search: "loja" });

// Exemplo de mutation no frontend
const createVisit = trpc.manutencao.createVisit.useMutation({
  onSuccess: () => utils.manutencao.listClients.invalidate()
});
```

O tRPC serializa automaticamente tipos complexos (Date, Map, etc.) via superjson. Todos os inputs são validados com Zod no servidor.

2.6 Armazenamento de Arquivos (S3)

Todos os arquivos binários são armazenados no AWS S3 via helper `storagePut()`. O banco de dados armazena apenas a URL pública e a chave S3.

Tipos de arquivos armazenados:

- **PDFs de relatório de preventiva** (`reportPdfUrl` em `maintenanceVisits`)
 - **Assinaturas digitais** (`signatureUrl` em `maintenanceVisits`)
 - **Fotos do checklist** (embutidas no PDF via base64, não armazenadas separadamente)
 - **PDFs de orçamento** (`fileUrl` em `maintenanceQuotes`)
 - **Comprovantes de pagamento** (`receiptUrl` em `materialRequests`)
 - **Fotos de chamados** (`photoUrl` em `maintenanceTickets`)
 - **Fotos de suporte** (`imageUrl` em `supportTickets`)
 - **Documentos institucionais** (`fileUrl` em `institutionalDocuments`)
 - **Vídeos de treinamento** (`videoUrl` em `trainingVideos`)
-

Portal CJR — Documentação Completa

Documento 03: Modelagem de Dados

3.1 Visão Geral do Banco de Dados

O banco de dados é MySQL/TiDB gerenciado via Drizzle ORM. O schema está definido em `drizzle/schema.ts` . As tabelas estão organizadas em dois grupos principais: **módulo do fórum/plataforma interna** e **módulo de gestão de manutenção**.

3.2 Tabelas do Módulo de Manutenção

`maintenanceClients` — Clientes de Manutenção

Cada registro representa **uma loja** de um cliente. Múltiplas lojas do mesmo grupo compartilham o mesmo `clientCode` .

Campo	Tipo	Descrição
id	INT PK	Identificador único
companyName	VARCHAR(255)	Razão social da empresa
responsibleName	VARCHAR(255)	Nome do responsável
cnpj	VARCHAR(20)	CNPJ da empresa
phone	VARCHAR(30)	Telefone
whatsapp	VARCHAR(30)	WhatsApp
email	VARCHAR(320)	E-mail (usado para login)
storeName	VARCHAR(255)	Nome da loja
shoppingName	VARCHAR(255)	Nome do shopping
address	TEXT	Endereço completo
storeArea	DECIMAL(10,2)	Área da loja em m²
storeType	ENUM	Tipo: quiosque , ate_40 , 41_80 , 81_120 , 120_250
planType	ENUM	Plano: trimestral , semestral , anual
hasEletrica	BOOLEAN	Serviço elétrico incluído
hasHidraulica	BOOLEAN	Serviço hidráulico incluído
hasCivil	BOOLEAN	Serviço civil incluído
monthlyValue	DECIMAL(10,2)	Valor mensal calculado
totalPlanValue	DECIMAL(10,2)	Valor total do plano
contractStatus	ENUM	Status: ativo , pendente , inadimplente , cancelado , suspenso
contractStartDate	TIMESTAMP	Data de início do contrato
contractEndDate	TIMESTAMP	Data de término do contrato
clientCode	VARCHAR(20) UNIQUE	Código do cliente (ex.: CLT-0001) — compartilhado entre lojas do mesmo grupo
storeNumber	INT	Número sequencial da loja dentro do grupo
storeCode	VARCHAR(30) UNIQUE	Código único da loja (ex.: CLT-0001-L2)

Campo	Tipo	Descrição
<code>storeApprovalStatus</code>	ENUM	Aprovação: <code>pendente_aprovacao</code> , <code>aprovado</code> , <code>rejeitado</code>
<code>userId</code>	INT	FK → <code>users.id</code> (opcional, para vinculação OAuth)
<code>passwordHash</code>	VARCHAR(255)	Hash bcrypt da senha de acesso
<code>mustChangePassword</code>	BOOLEAN	Obriga troca de senha no próximo login
<code>passwordResetToken</code>	VARCHAR(128)	Token de redefinição de senha
<code>passwordResetExpires</code>	TIMESTAMP	Expiração do token de reset
<code>notes</code>	TEXT	Observações internas
<code>createdById</code>	INT	FK → <code>users.id</code> (quem cadastrou)
<code>createdByName</code>	VARCHAR(255)	Nome de quem cadastrou
<code>createdAt</code>	TIMESTAMP	Data de criação
<code>updatedAt</code>	TIMESTAMP	Data de atualização

Regras de negócio:

- Um cliente pode ter múltiplas lojas; todas compartilham o mesmo `clientCode` e `email`
- O login do cliente usa `email` + `passwordHash` (bcrypt)
- `storeApprovalStatus` = "pendente_aprovacao" quando uma nova loja é adicionada pelo cliente; admin precisa aprovar
- Quando `contractStatus` muda para "ativo" , o sistema pode gerar automaticamente o cronograma de preventivas

maintenanceTechnicians — Técnicos de Manutenção

Campo	Tipo	Descrição
id	INT PK	Identificador único
userId	INT	FK → users.id (opcional)
name	VARCHAR(255)	Nome completo
phone	VARCHAR(30)	Telefone
email	VARCHAR(320)	E-mail (usado para login)
cpf	VARCHAR(20)	CPF
photoUrl	TEXT	URL da foto de perfil
specialties	TEXT	JSON array: ["eletrica","hidraulica","civil"]
isActive	BOOLEAN	Técnico ativo no sistema
passwordHash	VARCHAR(255)	Hash bcrypt da senha
mustChangePassword	BOOLEAN	Obriga troca de senha
passwordResetToken	VARCHAR(128)	Token de reset
passwordResetExpires	TIMESTAMP	Expiração do token
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

maintenanceVisits — Visitas Preventivas/Corretivas

Campo	Tipo	Descrição
id	INT PK	Identificador único
clientId	INT	FK → maintenanceClients.id
clientName	VARCHAR(255)	Nome do cliente (desnormalizado)
technicianId	INT	FK → maintenanceTechnicians.id
technicianName	VARCHAR(255)	Nome do técnico (desnormalizado)
visitType	ENUM	Tipo: preventiva , corretiva , instalacao , vistoria
scheduledDate	TIMESTAMP	Data agendada
scheduledTime	VARCHAR(10)	Hora agendada (ex.: "09:00")
checkInAt	TIMESTAMP	Hora de entrada do técnico
checkOutAt	TIMESTAMP	Hora de saída do técnico
status	ENUM	Status: agendada , em_andamento , concluida , cancelada , reagendada
checklistItems	TEXT	JSON array de {label, done, photoUrl} — 36 itens
techNotes	TEXT	Observações do técnico
servicesPerformed	TEXT	JSON array de serviços realizados
photoBefore	TEXT	JSON array de URLs de fotos (antes)
photoAfter	TEXT	JSON array de URLs de fotos (depois)
clientSignatureUrl	VARCHAR(1024)	URL da assinatura digital do cliente
clientSignatureName	VARCHAR(255)	Nome de quem assinou
signatureUrl	VARCHAR(1024)	URL alternativa de assinatura
reportPdfUrl	VARCHAR(1024)	URL do PDF do relatório gerado
reportPdfKey	VARCHAR(512)	Chave S3 do PDF
createdById	INT	FK → users.id (quem agendou)
createdAt	TIMESTAMP	Data de criação

Campo	Tipo	Descrição
<code>updatedAt</code>	TIMESTAMP	Data de atualização

Regras de negócio:

- O PDF do relatório é gerado no frontend (jspdf) com logo CJR, dados da visita, fotos do checklist e assinatura digital
 - O PDF é enviado ao servidor como base64 e salvo no S3 via `storagePut()`
 - Visitas geradas automaticamente pelo cronograma têm `adminNotes` com o texto `"Preventiva N/Total – gerada automaticamente"`
-

maintenanceTickets — Chamados de Manutenção

Campo	Tipo	Descrição
id	INT PK	Identificador único
clientId	INT	FK → maintenanceClients.id
title	VARCHAR(255)	Título do chamado
description	TEXT	Descrição do problema
category	ENUM	Categoria: eletrica , hidraulica , civil , ar_condicionado , equipamentos , outro
priority	ENUM	Prioridade: baixa , media , alta , urgente
status	ENUM	Status: aberto , em_andamento , aguardando_material , concluido , cancelado
technicianId	INT	FK → maintenanceTechnicians.id
technicianName	VARCHAR(255)	Nome do técnico (desnormalizado)
visitId	INT	FK → maintenanceVisits.id
location	VARCHAR(500)	Localização do problema na loja
photoUrl	VARCHAR(1024)	URL da foto do problema
photoUrls	TEXT	JSON array de URLs adicionais
resolution	TEXT	Descrição da resolução
resolvedAt	TIMESTAMP	Data de resolução
openedById	INT	FK → usuário que abriu
openedByName	VARCHAR(255)	Nome de quem abriu
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

materialRequests — Solicitações de Material pelo Técnico

Campo	Tipo	Descrição
id	INT PK	Identificador único
ticketId	INT	FK → maintenanceTickets.id (opcional)
visitId	INT	FK → maintenanceVisits.id (opcional)
clientId	INT	FK → maintenanceClients.id
technicianId	INT	FK → maintenanceTechnicians.id
technicianName	VARCHAR(255)	Nome do técnico
description	VARCHAR(500)	Descrição do material
estimatedValue	DECIMAL(10,2)	Valor estimado
status	ENUM	Status: pendente , aprovado , rejeitado , pago
clientApprovedAt	TIMESTAMP	Data de aprovação pelo cliente
clientRejectedAt	TIMESTAMP	Data de rejeição pelo cliente
clientNotes	VARCHAR(500)	Observações do cliente
paymentLink	VARCHAR(1024)	Link de pagamento Mercado Pago
paymentPreferenceId	VARCHAR(255)	ID da preferência Mercado Pago
paidAt	TIMESTAMP	Data de pagamento
receiptUrl	VARCHAR(1024)	URL do comprovante de pagamento
receiptKey	VARCHAR(512)	Chave S3 do comprovante
receiptUploadedAt	TIMESTAMP	Data de upload do comprovante
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

maintenanceQuotes — Orçamentos de Serviços Extras

Campo	Tipo	Descrição
id	INT PK	Identificador único
clientId	INT	FK → maintenanceClients.id
title	VARCHAR(255)	Título do orçamento
description	TEXT	Descrição do serviço
value	DECIMAL(10,2)	Valor do orçamento
status	ENUM	Status: em_analise , em_ajuste , aprovado , rejeitado , executado , respondido
fileUrl	VARCHAR(1024)	URL do PDF do orçamento
fileKey	VARCHAR(512)	Chave S3 do PDF
clientFeedback	TEXT	Feedback do cliente
clientSuggestion	TEXT	Sugestão de ajuste do cliente
roundCount	INT	Número de rodadas de negociação (default: 1)
approvedAt	TIMESTAMP	Data de aprovação
rejectedAt	TIMESTAMP	Data de rejeição
adminResponse	TEXT	Resposta do admin ao cliente
deadline	VARCHAR(100)	Prazo de execução
respondedAt	TIMESTAMP	Data de resposta do admin
createdById	INT	FK → users.id (admin que criou)
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

Regras de negócio do fluxo de negociação:

- 1. Admin cria orçamento com PDF → status "em_analise"
- 2. Cliente aprova → status "aprovado" , approvedAt preenchido
- 3. Cliente rejeita → status "rejeitado" , rejectedAt preenchido
- 4. Cliente sugere ajuste → clientSuggestion preenchido, status "em_ajuste" , roundCount incrementado

5. Admin vê sugestão e reenvia novo PDF → status volta para "em_analise" , nova rodada
6. Ciclo se repete até aprovação ou rejeição definitiva

preventiveConfigs — Configuração de Preventiva por Loja

Campo	Tipo	Descrição
id	INT PK	Identificador único
clientId	INT UNIQUE	FK → maintenanceClients.id
firstPreventiveDate	TIMESTAMP	Data da primeira preventiva
nextPreventiveDate	TIMESTAMP	Próxima preventiva calculada
intervalDays	INT	Intervalo padrão em dias (default: 30)
notes	TEXT	Observações do admin
updatedByName	VARCHAR(255)	Nome de quem atualizou
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

preventiveDateHistory — Histórico de Alterações de Data

Campo	Tipo	Descrição
id	INT PK	Identificador único
clientId	INT	FK → maintenanceClients.id
previousDate	TIMESTAMP	Data anterior
newDate	TIMESTAMP	Nova data
changedByName	VARCHAR(255)	Nome de quem alterou
reason	VARCHAR(500)	Motivo da alteração
createdAt	TIMESTAMP	Data do registro

supportTickets — Chamados de Suporte

Campo	Tipo	Descrição
id	INT PK	Identificador único
openedByType	ENUM	Quem abriu: tecnico ou admin
openedById	INT	ID de quem abriu
openedByName	VARCHAR(255)	Nome de quem abriu
description	TEXT	Descrição do problema
imageUrl	VARCHAR(1024)	URL da imagem anexada
imageKey	VARCHAR(512)	Chave S3 da imagem
status	ENUM	Status: aberto , em_andamento , resolvido , fechado
adminNotes	TEXT	Notas do admin na resolução
resolvedAt	TIMESTAMP	Data de resolução
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

clientSessions — Sessões de Cliente

Campo	Tipo	Descrição
id	INT PK	Identificador único
clientId	INT	FK → maintenanceClients.id
token	VARCHAR(255) UNIQUE	Token de sessão (96 hex chars)
expiresAt	TIMESTAMP	Expiração da sessão
createdAt	TIMESTAMP	Data de criação

technicianSessions — Sessões de Técnico

Campo	Tipo	Descrição
id	INT PK	Identificador único
technicianId	INT	FK → maintenanceTechnicians.id
token	VARCHAR(255) UNIQUE	Token de sessão
expiresAt	TIMESTAMP	Expiração da sessão
createdAt	TIMESTAMP	Data de criação

technicianClientLinks — Vínculo Técnico ↔ Cliente

Campo	Tipo	Descrição
id	INT PK	Identificador único
technicianId	INT	FK → maintenanceTechnicians.id
clientId	INT	FK → maintenanceClients.id
createdAt	TIMESTAMP	Data de criação

Regra: Um técnico só vê as lojas (clientes) às quais está vinculado via esta tabela.

maintenanceMaterials — Materiais Aprovados (legado)

Tabela original de materiais (anterior à materialRequests). Mantida por compatibilidade.

Campo	Tipo	Descrição
id	INT PK	Identificador único
visitId	INT	FK → maintenanceVisits.id
ticketId	INT	FK → maintenanceTickets.id
clientId	INT	FK → maintenanceClients.id
description	VARCHAR(500)	Descrição
estimatedValue	DECIMAL(10,2)	Valor estimado
finalValue	DECIMAL(10,2)	Valor final
adminFee	DECIMAL(10,2)	Taxa administrativa (30%)
invoiceUrl	VARCHAR(1024)	URL da nota fiscal
status	ENUM	Status: pendente , aprovado , reprovado , comprado
approvedById	INT	FK → users.id
approvedAt	TIMESTAMP	Data de aprovação
requestedById	INT	FK → users.id
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

`maintenancePriceConfig` — Configuração de Preços

Campo	Tipo	Descrição
id	INT PK	Identificador único
storeType	VARCHAR(50) UNIQUE	Tipo de loja
basePrice	INT	Preço base (default: 280)
additionalServiceCost	INT	Custo por serviço adicional (default: 65)
updatedAt	TIMESTAMP	Data de atualização
updatedByName	VARCHAR(255)	Nome de quem atualizou

maintenanceLeads — Pré-cadastros / Leads

Campo	Tipo	Descrição
id	INT PK	Identificador único
name	VARCHAR(255)	Nome do responsável
phone	VARCHAR(30)	Telefone
email	VARCHAR(320)	E-mail
storeName	VARCHAR(255)	Nome da loja
shoppingName	VARCHAR(255)	Nome do shopping
storeType	ENUM	Tipo de loja
hasEletrica/Hidraulica/Civil	BOOLEAN	Serviços desejados
planType	ENUM	Plano escolhido
monthlyValue	DECIMAL(10,2)	Valor mensal simulado
totalValue	DECIMAL(10,2)	Valor total simulado
cnpj	VARCHAR(20)	CNPJ
companyName	VARCHAR(255)	Razão social
address	TEXT	Endereço
responsibleName	VARCHAR(255)	Nome do responsável
whatsapp	VARCHAR(30)	WhatsApp
leadStep	INT	Etapa: 1=pré-cadastro, 2=simulação, 3=cadastro, 4=contrato, 5=pagamento
leadStatus	ENUM	Status: novo , em_andamento , convertido , perdido , contato_feito
contractAccepted	BOOLEAN	Contrato aceito
signatureName	VARCHAR(255)	Nome na assinatura do contrato
paymentPreferenceId	VARCHAR(255)	ID Mercado Pago
paymentStatus	ENUM	Status do pagamento
prospectStatus	ENUM	Status de prospecção manual

Campo	Tipo	Descrição
<code>clientId</code>	INT	FK → <code>maintenanceClients.id</code> (quando convertido)
<code>notes</code>	TEXT	Observações
<code>lastContactAt</code>	TIMESTAMP	Último contato
<code>createdAt</code>	TIMESTAMP	Data de criação
<code>updatedAt</code>	TIMESTAMP	Data de atualização

`maintenanceCoupons` — Cupons de Desconto

Campo	Tipo	Descrição
<code>id</code>	INT PK	Identificador único
<code>code</code>	VARCHAR(50) UNIQUE	Código do cupom
<code>description</code>	VARCHAR(255)	Descrição
<code>discountPercent</code>	DECIMAL(5,2)	Percentual de desconto (ex.: 15.00 = 15%)
<code>discountMonths</code>	INT	Por quantos meses o desconto se aplica
<code>maxUses</code>	INT	Máximo de usos (null = ilimitado)
<code>usedCount</code>	INT	Quantidade de usos
<code>validFrom</code>	TIMESTAMP	Início da validade
<code>validUntil</code>	TIMESTAMP	Fim da validade
<code>applicablePlans</code>	VARCHAR(100)	Planos aplicáveis (null = todos)
<code>isActive</code>	BOOLEAN	Cupom ativo
<code>createdByName</code>	VARCHAR(255)	Nome de quem criou
<code>createdAt</code>	TIMESTAMP	Data de criação
<code>updatedAt</code>	TIMESTAMP	Data de atualização

systemSettings — Configurações do Sistema

Campo	Tipo	Descrição
id	INT PK	Identificador único
key	VARCHAR(100) UNIQUE	Chave da configuração (ex.: "contract_text")
value	TEXT	Valor da configuração
label	VARCHAR(255)	Nome amigável para o admin
updatedAt	TIMESTAMP	Data de atualização
updatedAtName	VARCHAR(255)	Nome de quem atualizou

actionLogs — Logs de Ações do Sistema

Campo	Tipo	Descrição
id	INT PK	Identificador único
entityType	VARCHAR(50)	Tipo: client , technician , coupon , payment , lead
entityId	INT	ID da entidade
entityCode	VARCHAR(50)	Código da entidade (ex.: CLT-0001)
action	VARCHAR(100)	Ação: created , updated , deleted , activated , suspended
description	TEXT	Descrição da ação
performedById	INT	FK → users.id
performedByName	VARCHAR(255)	Nome de quem realizou
createdAt	TIMESTAMP	Data do log

3.3 Tabelas do Módulo de Fórum/Plataforma Interna

Tabela	Descrição
users	Usuários internos (colaboradores CJR) com autenticação OAuth
documents	Documentos da base de conhecimento
conversations	Conversas com a IA
messages	Mensagens individuais das conversas
knowledgeBase	Entradas da base de conhecimento
notifications	Notificações do sistema
deliveryTerms	Termos de entrega gerados
institutionalDocuments	Documentos institucionais (onboarding)
meetingMinutes	Atas de reunião
trainingVideos	Vídeos de treinamento
trainingPdfs	PDFs de treinamento
inventoryItems	Itens de estoque
inventoryMovements	Movimentações de estoque
equipment	Equipamentos
prospectClients	Clientes em prospecção (CRM)
prospectContacts	Contatos dos prospects
prospectInteractions	Histórico de interações
prospectTasks	Tarefas internas
emailTemplates	Templates de e-mail
followupRules	Regras de follow-up automático
commercialStandards	Padrões comerciais

3.4 Diagrama de Relacionamentos (Módulo de Manutenção)

```
maintenanceClients (1) ————— (N) maintenanceVisits
maintenanceClients (1) ————— (N) maintenanceTickets
maintenanceClients (1) ————— (N) materialRequests
maintenanceClients (1) ————— (N) maintenanceQuotes
maintenanceClients (1) ————— (N) maintenanceMaterials
maintenanceClients (1) ————— (1) preventiveConfigs
maintenanceClients (1) ————— (N) preventiveDateHistory
maintenanceClients (1) ————— (N) clientSessions

maintenanceTechnicians (1) ————— (N) maintenanceVisits
maintenanceTechnicians (1) ————— (N) materialRequests
maintenanceTechnicians (1) ————— (N) technicianClientLinks
maintenanceTechnicians (1) ————— (N) technicianSessions
maintenanceTechnicians (1) ————— (N) supportTickets

maintenanceVisits (1) ————— (N) materialRequests
maintenanceTickets (1) ————— (N) materialRequests

users (1) ————— (N) maintenanceClients (createdById)
users (1) ————— (N) maintenanceVisits (createdById)
users (1) ————— (N) maintenanceQuotes (createdById)
```

Portal CJR — Documentação Completa

Documento 04: Documentação das Telas

4.1 Mapa de Rotas

Rota	Componente	Acesso	Descrição
/	Home	Público	Hub de navegação / landing page
/login-cliente	ClienteLogin	Público	Login do cliente
/login-tecnico	TecnicoLogin	Público	Login do técnico
/cliente/definir-senha	ClienteDefinirSenha	Token URL	Definir senha inicial (cliente)
/tecnico/definir-senha	TecnicoDefinirSenha	Token URL	Definir senha inicial (técnico)
/cliente/trocar-senha	ClienteTrocarSenha	Autenticado	Trocar senha (cliente)
/tecnico/trocar-senha	TecnicoTrocarSenha	Autenticado	Trocar senha (técnico)
/cliente	ClientePortal	Cliente autenticado	Portal completo do cliente
/tecnico	TecnicoPortal	Técnico autenticado	Portal completo do técnico
/manutencao/contratar	ContratarManutencao	Público	Fluxo de contratação online
/manutencao/dashboard	DashboardManutencao	Usuário interno	Dashboard de manutenção
/manutencao/clientes	ClientesManutencao	Usuário interno	Listagem de clientes
/manutencao/clientes/:id	ClienteDetalhe	Usuário interno	Detalhe do cliente

Rota	Componente	Acesso	Descrição
/manutencao/chamados	ChamadosManutencao	Usuário interno	Gestão de chamados
/manutencao/agenda	AgendaManutencao	Usuário interno	Agenda de visitas
/manutencao/tecnicos	TecnicosManutencao	Admin	Gestão de técnicos
/manutencao/preventivos	PreventivosManutencao	Admin	Gestão de preventivos
/manutencao/orçamentos	OrcamentosManutencao	Admin	Gestão de orçamentos
/admin/manutencao	AdminManutencao	Admin	Painel administrativo completo
/admin/manutencao/clientes/:id	AdminClienteDetalhe	Admin	Detalhe do cliente (admin)

4.2 Telas do Portal do Cliente (/cliente)

Tela: Login do Cliente (/login-cliente)

Objetivo: Autenticar o cliente (lojista) no portal.

Campos:

- E-mail (obrigatório)
- Senha (obrigatório, mínimo 6 caracteres)

Botões e Ações:

- Entrar — chama `clientAuth.clientLogin` ; em caso de sucesso, redireciona para /cliente
- Esqueci minha senha — redireciona para fluxo de reset (não implementado na UI, mas disponível via admin)

Validações:

- E-mail e senha são obrigatórios
- Se `mustChangePassword = true` , redireciona para /cliente/trocar-senha após login

Navegação: Após login bem-sucedido → `/cliente`

Tela: Portal do Cliente (`/cliente`)

Objetivo: Central de acesso do cliente para acompanhar todos os serviços de manutenção.

Seções principais (menu lateral/tabs):

- `dashboard` — Visão geral com gráfico de chamados
- `lojas` — Listagem e detalhes das lojas
- `plano` — Informações do plano contratado
- `termo` — Contrato de adesão

Seção Dashboard:

- Gráfico de pizza (recharts) com chamados por categoria no ano atual
- Cards com contadores: total de chamados, chamados abertos, preventivas realizadas

Seção Lojas (por loja selecionada): Tabs internas:

- `chamados` — Lista de chamados com status, categoria, prioridade, foto e localização
 - Botão `+ Novo Chamado` → modal com campos: título, descrição, categoria, prioridade, localização, foto
 - Botão `Aprovar` / `Rejeitar` em chamados pendentes de material
- `preventivas` — Histórico de visitas preventivas com status e link para PDF do relatório
 - Card com próxima preventiva (badge colorido por urgência)
- `checklist` — Visualização do checklist da última preventiva
- `orcamentos` — Lista de orçamentos com status e fluxo de negociação
 - Botão `Aprovar` → chama `clientAuth.clientRespondQuote` com `action: "aprovado"`
 - Botão `Rejeitar` → modal com campo de feedback → `action: "rejeitado"`
 - Botão `Sugerir Ajuste` → modal com campo de sugestão → `action: "ajuste"`, preenche `clientSuggestion`
 - Botão `Ver PDF` → abre URL do PDF em nova aba
- `materiais` — Lista de solicitações de material
 - Botão `Aprovar` / `Rejeitar` em solicitações pendentes
 - Botão `Enviar Comprovante` (após aprovação) → modal com upload de imagem/PDF → `clientAuth.clientUploadMaterialReceipt`
 - Botão `Ver Comprovante` (após upload) → abre URL

Seção Plano:

- Exibe tipo de plano, valor mensal, serviços incluídos, datas de início/fim

Seção Termo:

- Exibe o texto do contrato de adesão (carregado de `systemSettings` com chave `"contract_text"`)

Redirecionamentos:

- Se não autenticado → `/login-cliente`
 - Se `mustChangePassword = true` → `/cliente/trocar-senha`
-

4.3 Telas do Portal do Técnico (`/tecnico`)

Tela: Login do Técnico (`/login-tecnico`)

Objetivo: Autenticar o técnico no portal.

Campos:

- E-mail (obrigatório)
- Senha (obrigatório)

Botões: Entrar — chama `clientAuth.techLogin`

Navegação: Após login → `/tecnico`

Tela: Portal do Técnico (`/tecnico`)

Objetivo: Interface de trabalho do técnico de campo.

Tabs principais:

- `lojas` — Lojas vinculadas ao técnico
- `agenda` — Agenda de visitas agendadas
- `chamados` — Chamados atribuídos ao técnico

Tab Lojas:

- Lista de lojas vinculadas via `technicianClientLinks`
- Por loja, sub-tabs: `chamados`, `preventivas`, `checklist`, `materiais`

Tab Agenda:

- Calendário/lista de visitas agendadas
- Badge colorido com próxima preventiva
- Clique no badge → modal com lista de lojas com preventiva no mesmo dia

Tab Chamados:

- Lista de chamados atribuídos
- Botão **Executar** → abre painel de execução:
 - Atualização de status
 - Foto obrigatória ao concluir
 - Visualização da foto/localização do problema
 - Botão **Solicitar Material** → modal com campos: descrição, valor estimado

Execução de Visita Preventiva (Checklist):

Ao iniciar uma visita preventiva, o técnico acessa o checklist de 36 itens divididos em 3 grupos:

CIVIL (12 itens):

1. Verificação de pisos e revestimentos
2. Inspeção de paredes e forros
3. Verificação de portas e janelas
4. Inspeção de calhas e rufos
5. Verificação de impermeabilização
6. Inspeção de estrutura metálica
7. Verificação de pintura
8. Inspeção de vidros e esquadrias
9. Verificação de fechaduras e ferragens
10. Inspeção de sinalização
11. Verificação de acessibilidade
12. Inspeção geral de acabamentos

HIDRÁULICA (12 itens):

1. Verificação de torneiras e registros
2. Inspeção de vasos sanitários
3. Verificação de pias e cubas
4. Inspeção de ralos e sifões
5. Verificação de caixas d'água
6. Inspeção de bombas d'água
7. Verificação de aquecedores
8. Inspeção de tubulações aparentes
9. Verificação de pressão da água
10. Inspeção de válvulas de descarga
11. Verificação de chuveiros
12. Inspeção de sistema de esgoto

ELÉTRICA (12 itens):

1. Verificação de quadro elétrico
2. Inspeção de disjuntores
3. Verificação de tomadas e interruptores
4. Inspeção de iluminação
5. Verificação de aterramento
6. Inspeção de fiação aparente
7. Verificação de ar-condicionado
8. Inspeção de nobreaks e estabilizadores
9. Verificação de câmeras de segurança
10. Inspeção de alarmes
11. Verificação de SPDA (para-raios)
12. Inspeção geral de equipamentos elétricos

Para cada item:

- Checkbox de conclusão
- Botão de câmera → captura foto (obrigatória para itens marcados como concluídos)
- A foto é armazenada em base64 no estado local e incluída no PDF

Finalização da Visita:

1. Técnico preenche observações gerais
2. Tela de assinatura digital (canvas com `signature_pad`)
3. Campo para nome do assinante
4. Botão Gerar Relatório e Finalizar :
 - Gera PDF com jsPDF (logo CJR, dados, checklist, fotos, assinatura)
 - Envia PDF como base64 para o servidor via `clientAuth.techExecuteVisit`
 - Servidor salva PDF no S3 e atualiza `maintenanceVisits.reportPdfUrl`

Tab Suporte:

- Lista de chamados de suporte abertos pelo técnico
 - Botão + Novo Chamado de Suporte → modal com campos: descrição, foto opcional
 - Chama `clientAuth.techCreateSupportTicket`
-

4.4 Telas do Painel Administrativo

Tela: Painel Admin de Manutenção (`/admin/manutencao`)

Objetivo: Central de gestão completa do módulo de manutenção para a equipe CJR.

Tabs:

- `dashboard` — Métricas gerais (total de clientes, visitas, chamados, receita)
- `clientes` — Listagem e gestão de clientes
- `leads` — Pré-cadastros e prospecção
- `tecnicos` — Gestão de técnicos
- `equipe` — Controle de acesso da equipe interna
- `precos` — Configuração de tabela de preços
- `cupons` — Gestão de cupons de desconto
- `contrato` — Edição do texto do contrato de adesão
- `orcamentos` — Listagem de todos os orçamentos
- `suporte` — Chamados de suporte dos técnicos

Tab Clientes:

- Cards compactos com código, nome da empresa, loja, status do contrato
- Botão `Ver Detalhes` → navega para `/admin/manutencao/clientes/:id`
- Botão `+ Novo Cliente` → modal com formulário completo de cadastro
- Filtros por status do contrato
- Exportação CSV

Tab Orçamentos:

- Cards de orçamentos com status, cliente, valor, rodada de negociação
- Filtros por status
- Botão `Responder` → modal com campos: texto de resposta, valor, prazo, upload de PDF
- Botão `+ Novo Orçamento` → modal com campos: cliente, título, descrição, valor, upload de PDF
- Chama `clientAuth.adminRespondQuote` (resposta) ou `manutencao.adminUploadQuote` (novo)

Tab Suporte:

- Lista de chamados de suporte com status
 - Botão `Responder/Fechar` → modal com campo de notas e mudança de status
-

Tela: Detalhe do Cliente — Admin (/admin/manutencao/clientes/:id)

Objetivo: Visão consolidada de todos os dados de um cliente específico.

6 Abas:

Aba Lojas:

- Lista todas as lojas do cliente (mesmo `clientId`)
- Status de aprovação de cada loja
- Botão `Aprovar` / `Rejeitar` para lojas pendentes

Aba Preventivas:

- Lista de visitas preventivas agendadas e concluídas
- Botão `+ Agendar Visita` → modal com campos: data, hora, técnico
- Botão `Cronograma Automático` → modal com:
 - Data de início (editável)
 - Preview das datas geradas (1ª visita = data informada, 2ª = +10 dias, demais = +30 dias)
 - Botão `Confirmar e Gerar` → chama `manutencao.generatePreventiveSchedule`

Aba Relatórios:

- Lista de visitas concluídas com PDF disponível
- Botão `Ver Relatório` → abre PDF em nova aba

Aba Materiais:

- Lista de solicitações de material com status
- Botão `Aprovar` / `Reprovar` → chama `manutencao.approveMaterial`

Aba Financeiro:

- Resumo financeiro: total de manutenção, total de materiais, total geral
- Calculado pela procedure `adminGetClientDetail`

Aba Orçamentos:

- Lista de orçamentos do cliente
- Botão `Responder` → modal de resposta com upload de PDF

Tela: Contratação Online (/manutencao/contratar)

Objetivo: Fluxo público de contratação de plano de manutenção.

Etapas:

1. **Dados da Loja:** nome, shopping, tipo de loja, serviços desejados

2. **Simulação:** exibe preço calculado por plano (trimestral/semestral/anual) com desconto
3. **Dados Completos:** CNPJ, razão social, endereço, responsável, WhatsApp
4. **Contrato:** exibe texto do contrato, campo de assinatura (nome), checkbox de aceite
5. **Pagamento:** integração Mercado Pago (link de pagamento gerado)

Validações:

- Todos os campos obrigatórios devem ser preenchidos antes de avançar
 - Cupom de desconto pode ser aplicado na etapa de simulação
 - Contrato deve ser aceito antes de prosseguir para pagamento
-

4.5 Telas do Módulo de Gestão (Usuários Internos)

Tela: Dashboard de Manutenção (/manutencao/dashboard)

Objetivo: Visão geral operacional do módulo de manutenção.

Conteúdo:

- Cards com KPIs: total de clientes ativos, visitas do mês, chamados abertos, receita mensal
 - Gráficos de tendência
-

Tela: Listagem de Clientes (/manutencao/clientes)

Objetivo: Listar e buscar clientes de manutenção.

Campos de busca: nome da empresa, nome da loja, responsável

Filtros: status do contrato

Ações: Ver detalhes, exportar CSV

Tela: Orçamentos (/manutencao/orcamentos)

Objetivo: Gerenciar todos os orçamentos de serviços extras.

Campos de busca: cliente, título

Filtros: status (em análise, em ajuste, aprovado, rejeitado, executado)

Cards de resumo: total por status

Ações: Responder orçamento (com upload de PDF), criar novo orçamento

Portal CJR — Documentação Completa

Documento 05: Documentação das APIs (tRPC)

5.1 Visão Geral

O Portal CJR utiliza **tRPC v11** para comunicação frontend ↔ backend. Não há endpoints REST manuais para funcionalidades de negócio. Todas as chamadas são feitas via `/api/trpc/{router}.{procedure}`.

Autenticação:

- Usuários internos (admin, coordenador, etc.): cookie de sessão OAuth (`manus_session`)
- Clientes: cookie `cjr_client_session`
- Técnicos: cookie `cjr_tech_session`

Tipos de procedure:

- `publicProcedure` — Sem autenticação requerida
- `protectedProcedure` — Requer usuário interno autenticado (OAuth)
- Procedures de cliente/técnico verificam manualmente o cookie de sessão

5.2 Router: `manutencao` (Administração)

Base: `trpc.manutencao.*`

`calcPrice` — Calcular Preço do Plano

Tipo: Query (público)

Descrição: Calcula o valor mensal e total de um plano com base no tipo de loja e serviços.

Input:

```
{
  storeType: string;    // "quiosque" | "ate_40" | "41_80" | "81_120" | "120_250"
  planType: string;     // "trimestral" | "semestral" | "anual"
  hasEletrica: boolean;
  hasHidraulica: boolean;
  hasCivil: boolean;
}
```

Output:

```
{
  monthly: number; // Valor mensal com desconto do plano
  total: number; // Valor total do plano
  savings: number; // Economia em relação ao plano trimestral
}
```

listClients — Listar Clientes

Tipo: Query (protegido)

Descrição: Retorna todos os clientes de manutenção com filtros opcionais.

Input:

```
{
  search?: string; // Busca por nome da empresa, loja ou responsável
  status?: string; // Filtro por contractStatus
}
```

Output: Array de MaintenanceClient

getClient — Obter Cliente

Tipo: Query (protegido)

Input: { id: number }

Output: MaintenanceClient | null

createClient — Criar Cliente

Tipo: Mutation (protegido, admin)

Descrição: Cria um novo cliente de manutenção com código único gerado automaticamente.

Input:

```
{
  companyName: string;
  responsibleName: string;
  email: string;
  phone?: string;
  whatsapp?: string;
  cnpj?: string;
  storeName: string;
  shoppingName?: string;
  address?: string;
  storeType: string;
  planType: string;
  hasEletrica: boolean;
  hasHidraulica: boolean;
  hasCivil: boolean;
  contractStatus?: string;
  notes?: string;
}
```

Output: { id: number; clientCode: string }

updateClient — Atualizar Cliente

Tipo: Mutation (protegido)

Input: { id: number; ...campos opcionais }

Output: { ok: true }

createVisit — Agendar Visita

Tipo: Mutation (protegido)

Descrição: Cria uma nova visita preventiva ou corretiva.

Input:

```
{
  clientId: number;
  technicianId?: number;
  visitType?: string; // "preventiva" | "corretiva" | "instalacao" | "vistoria"
  scheduledDate: string; // ISO date string
  scheduledTime?: string; // "HH:MM"
  notes?: string;
}
```

Output: { id: number }

generatePreventiveSchedule — Gerar Cronograma Automático

Tipo: Mutation (protegido, admin)

Descrição: Gera automaticamente todas as visitas preventivas do plano a partir de uma data inicial.

Input:

```
{
  clientId: number;
  startDate?: string; // ISO date, default = hoje
}
```

Lógica:

- Plano trimestral: 3 visitas
- Plano semestral: 6 visitas
- Plano anual: 12 visitas
- 1ª visita: data informada
- 2ª visita: +10 dias
- Demais: +30 dias da anterior

Output: { success: true; count: number }

approveMaterial — Aprovar/Reprovar Material

Tipo: Mutation (protegido)

Input:

```
{
  materialId: number;
  action: "aprovado" | "reprovado";
  notes?: string;
}
```

Output: { ok: true }

adminUploadQuote — Criar Orçamento com PDF

Tipo: Mutation (protegido, admin)

Descrição: Cria um novo orçamento para um cliente e faz upload do PDF para o S3.

Input:

```
{
  clientId: number;
  title: string;
  description?: string;
  value?: number;
  fileBase64?: string;    // PDF em base64
  fileMime?: string;      // "application/pdf"
}
```

Output: { id: number }

getAdminDashboard — Dashboard Administrativo

Tipo: Query (protegido)

Output:

```
{
  totalClients: number;
  activeClients: number;
  totalVisits: number;
  visitsThisMonth: number;
  openTickets: number;
  monthlyRevenue: number;
}
```

getSetting — Obter Configuração do Sistema

Tipo: Query (público)

Input: { key: string }

Output: { value: string } | null

updateSetting — Atualizar Configuração do Sistema

Tipo: Mutation (protegido, admin)

Input: { key: string; value: string }

Output: { ok: true }

adminGetClientDetail — Detalhe Consolidado do Cliente

Tipo: Query (protegido)

Input: { clientId: number }

Output:

```
{
  client: MaintenanceClient;
  stores: MaintenanceClient[];           // Todas as lojas do mesmo clientCode
  visits: MaintenanceVisit[];
  materials: MaintenanceMaterial[];
  quotes: MaintenanceQuote[];
  preventiveConfig: PreventiveConfig | null;
  reports: MaintenanceVisit[];           // Visitas com reportPdfUrl
  financial: {
    totalMaintenance: number;
    totalMaterials: number;
    totalGeneral: number;
  };
}
```

listTechnicians — Listar Técnicos

Tipo: Query (protegido)

Output: Array de `MaintenanceTechnician`

createTechnician — Criar Técnico

Tipo: Mutation (protegido, admin)

Input: { name, email, phone?, cpf?, specialties? }

Output: { id: number }

linkTechnicianToClient — Vincular Técnico ao Cliente

Tipo: Mutation (protegido, admin)

Input: { technicianId: number; clientId: number }

Output: { ok: true }

listSupportTickets — Listar Chamados de Suporte

Tipo: Query (protegido)

Output: Array de `SupportTicket`

respondSupportTicket — Responder Chamado de Suporte

Tipo: Mutation (protegido, admin)

Input: { ticketId: number; notes: string; status: string }

Output: { ok: true }

5.3 Router: `clientAuth` (Portais de Cliente e Técnico)

Base: `trpc.clientAuth.*`

`clientLogin` — Login do Cliente

Tipo: Mutation (público)

Input: `{ email: string; password: string }`

Output: `{ ok: true; clientCode: string; mustChangePassword: boolean }`

Efeito colateral: Define cookie `cjr_client_session`

`clientLogout` — Logout do Cliente

Tipo: Mutation (público)

Output: `{ ok: true }`

Efeito colateral: Remove cookie `cjr_client_session`

`clientMe` — Dados do Cliente Logado

Tipo: Query (público — verifica cookie internamente)

Output:

```
{
  id: number;
  clientCode: string;
  companyName: string;
  responsibleName: string;
  email: string;
  mustChangePassword: boolean;
  stores: Array<{
    id: number;
    storeName: string;
    storeCode: string;
    address: string;
    contractStatus: string;
    storeApprovalStatus: string;
    planType: string;
    hasEletrica: boolean;
    hasHidraulica: boolean;
    hasCivil: boolean;
    monthlyValue: string;
    contractStartDate: Date;
    contractEndDate: Date;
  }>;
} | null
```

clientSetPassword — Definir Senha (Token)

Tipo: Mutation (público)

Input: { token: string; password: string }

Output: { ok: true }

clientChangePassword — Trocar Senha (Autenticado)

Tipo: Mutation (público — verifica cookie)

Input: { currentPassword: string; newPassword: string }

Output: { ok: true }

clientDashboardStats — Estatísticas do Dashboard

Tipo: Query (público — verifica cookie)

Output:

```
{
  totalTickets: number;
  openTickets: number;
  totalVisits: number;
  ticketsByCategory: Array<{ category: string; count: number }>;
}
```

clientListTickets — Listar Chamados do Cliente

Tipo: Query (público — verifica cookie)

Input: { storeId?: number }

Output: Array de MaintenanceTicket

clientCreateTicket — Criar Chamado

Tipo: Mutation (público — verifica cookie)

Input:


```
{
  storeId: number;
  title: string;
  description: string;
  category: string;
  priority?: string;
  location?: string;
  photoBase64?: string;
  photoMime?: string;
}
```

Output: { id: number }

clientListVisits — Listar Visitas

Tipo: Query (público — verifica cookie)

Input: { storeId?: number }

Output: Array de MaintenanceVisit

clientListQuotes — Listar Orçamentos

Tipo: Query (público — verifica cookie)

Output: Array de MaintenanceQuote

clientRespondQuote — Responder Orçamento

Tipo: Mutation (público — verifica cookie)

Input:

```
{
  quoteId: number;
  action: "aprovado" | "rejeitado" | "ajuste";
  feedback?: string;
  suggestion?: string; // Obrigatório se action = "ajuste"
}
```

Output: { ok: true }

clientListMaterialRequests — Listar Solicitações de Material

Tipo: Query (público — verifica cookie)

Output: Array de MaterialRequest

clientRespondMaterialRequest — Aprovar/Rejeitar Material

Tipo: Mutation (público — verifica cookie)

Input: { requestId: number; action: "aprovado" | "rejeitado"; notes?: string }

Output: { ok: true }

clientUploadMaterialReceipt — Enviar Comprovante de Pagamento

Tipo: Mutation (público — verifica cookie)

Input:

```
{
  materialRequestId: number;
  receiptBase64: string; // Imagem ou PDF em base64
  receiptMime: string;   // "image/jpeg" | "image/png" | "application/pdf"
}
```

Output: { ok: true; receiptUrl: string }

clientUpdateProfile — Atualizar Perfil

Tipo: Mutation (público — verifica cookie)

Input: { email?: string; phone?: string; whatsapp?: string }

Output: { ok: true }

techLogin — Login do Técnico

Tipo: Mutation (público)

Input: { email: string; password: string }

Output: { ok: true; technicianId: number; mustChangePassword: boolean }

Efeito colateral: Define cookie `cjr_tech_session`

techLogout — Logout do Técnico

Tipo: Mutation (público)

Output: { ok: true }

techMe — Dados do Técnico Logado

Tipo: Query (público — verifica cookie)

Output: { id, name, email, specialties, linkedClients: [...] } | null

techExecuteVisit — Executar Visita (Enviar Relatório)

Tipo: Mutation (público — verifica cookie)

Descrição: Finaliza uma visita preventiva com checklist, assinatura e PDF do relatório.

Input:

```
{
  visitId: number;
  checklistItems: Array<{ label: string; done: boolean; photoB64?: string }>;
  techNotes?: string;
  signatureB64: string;           // Assinatura digital em base64 (PNG)
  signatureName: string;         // Nome do assinante
  reportPdfB64: string;          // PDF completo em base64
  reportPdfMime: string;         // "application/pdf"
  servicesPerformed?: string[];
}
```

Output: { ok: true; reportPdfUrl: string }

techRequestMaterial — Solicitar Material

Tipo: Mutation (público — verifica cookie)

Input:

```
{
  visitId?: number;
  ticketId?: number;
  clientId: number;
  description: string;
  estimatedValue: number;
}
```

Output: { id: number }

techListMaterialRequests — Listar Solicitações de Material (Técnico)

Tipo: Query (público — verifica cookie)

Output: Array de MaterialRequest

techCreateSupportTicket — Abrir Chamado de Suporte

Tipo: Mutation (público — verifica cookie)

Input:

```
{
  description: string;
  imageBase64?: string;
  imageMime?: string;
}
```

Output: { id: number }

adminListQuotes — Listar Todos os Orçamentos (Admin)

Tipo: Query (protegido)

Input: { status?: string; search?: string }

Output: Array de `MaintenanceQuote` com dados do cliente

adminRespondQuote — Responder Orçamento (Admin)

Tipo: Mutation (protegido, admin)

Input:

```
{
  quoteId: number;
  status: string;
  adminResponse?: string;
  value?: number;
  deadline?: string;
  fileBase64?: string;    // Novo PDF em base64 (para reenvio após ajuste)
  fileMime?: string;
}
```

Output: { ok: true }

adminGenerateClientPasswordReset — Gerar Link de Reset de Senha

Tipo: Mutation (protegido, admin)

Input: { clientId: number }

Output: { resetUrl: string }

Validade do token: 24 horas

adminGenerateTechPasswordReset — Gerar Link de Reset de Senha (Técnico)

Tipo: Mutation (protegido, admin)

Input: { technicianId: number }

Output: { resetUrl: string }

5.4 Formato de Chamada tRPC

Query (GET)

```
GET /api/trpc/manutencao.listClients?input={"search":"loja"}
```

Mutation (POST)

```
POST /api/trpc/clientAuth.clientLogin
Content-Type: application/json
```

```
{"json": {"email": "cliente@email.com", "password": "senha123"}}
```

Batch (múltiplas chamadas)

O cliente tRPC agrupa automaticamente múltiplas queries em uma única requisição HTTP:

```
GET /api/trpc/manutencao.listClients,clientAuth.clientMe?batch=1&input=...
```

5.5 Tratamento de Erros

O tRPC retorna erros padronizados com código HTTP e mensagem:

Código tRPC	HTTP	Descrição
UNAUTHORIZED	401	Sessão inválida ou expirada
FORBIDDEN	403	Permissão insuficiente
NOT_FOUND	404	Recurso não encontrado
BAD_REQUEST	400	Input inválido (Zod validation)
INTERNAL_SERVER_ERROR	500	Erro interno do servidor

Exemplo de resposta de erro:

```
{
  "error": {
    "message": "E-mail ou senha inválidos.",
    "code": -32600,
    "data": {
      "code": "BAD_REQUEST",
      "httpStatus": 400
    }
  }
}
```

Portal CJR — Documentação Completa

Documento 06: Regras de Negócio

6.1 Precificação de Planos

Tabela de Preços Base (por tipo de loja)

Tipo de Loja	Preço Base Mensal
Quiosque	R\$ 250,00
Até 40 m²	R\$ 280,00
41 a 80 m²	R\$ 300,00
81 a 120 m²	R\$ 400,00
120 a 250 m²	R\$ 650,00

Serviços Adicionais

O primeiro serviço (Elétrica, Hidráulica ou Civil) já está incluído no preço base. Cada serviço adicional além do primeiro acrescenta **R\$ 65,00** ao valor mensal.

Exemplos:

- 1 serviço: preço base
- 2 serviços: preço base + R\$ 65,00
- 3 serviços: preço base + R\$ 130,00

Multiplicadores por Plano

Plano	Multiplicador	Meses	Desconto
Trimestral	1,00 (sem desconto)	3	0%
Semestral	0,96	6	4%
Anual	0,92	12	8%

Fórmula de Cálculo

```
monthly = round((base + (max(0, services - 1) * 65)) * multiplier)
total = monthly * planMonths
savings = (base + additionalCost) * planMonths - total
```

Exemplo prático: Loja de 41-80 m², 2 serviços, plano anual:

- Base: R\$ 300,00
- Adicional: R\$ 65,00
- Subtotal: R\$ 365,00
- Multiplicador anual: 0,92
- Mensal: R\$ 336,00 (arredondado)
- Total: R\$ 4.032,00
- Economia: R\$ 348,00

6.2 Cronograma Automático de Preventivas

Ao ativar um cliente, o admin pode gerar automaticamente o cronograma completo de visitas preventivas para o período do plano.

Regras de Geração

Plano	Total de Visitas	Frequência
Trimestral	3	1 por mês
Semestral	6	1 por mês
Anual	12	1 por mês

Regras de Espaçamento

- **1ª visita:** data informada pelo admin (ou data atual se não informada)
- **2ª visita:** 1ª visita + 10 dias
- **3ª visita em diante:** visita anterior + 30 dias

Dados Gerados por Visita

- `clientId` : ID do cliente
 - `scheduledDate` : data calculada
 - `scheduledTime` : "09:00" (padrão)
 - `status` : "agendada"
 - `adminNotes` : "Preventiva N/Total – gerada automaticamente (plano TIPO)"
-

6.3 Fluxo de Aprovação de Orçamentos

O fluxo de negociação de orçamentos é **iterativo** e suporta múltiplas rodadas.

Estados do Orçamento

```
em_analise → aprovado (cliente aprova)
em_analise → rejeitado (cliente rejeita)
em_analise → em_ajuste (cliente sugere ajuste)
em_ajuste → em_analise (admin reenvia PDF com ajuste)
em_analise → executado (admin marca como executado)
```

Regras

1. Apenas o admin pode criar orçamentos e fazer upload de PDFs
 2. Apenas o cliente pode aprovar, rejeitar ou sugerir ajuste
 3. Quando o cliente sugere ajuste, o campo `clientSuggestion` é preenchido e `roundCount` é incrementado
 4. O admin vê a sugestão do cliente e pode reenviar um novo PDF (nova rodada)
 5. Não há limite de rodadas de negociação
 6. Uma vez aprovado ou rejeitado, o orçamento não pode ser reaberto (apenas o admin pode alterar o status manualmente)
-

6.4 Fluxo de Solicitação de Material

Estados da Solicitação

```
pendente → aprovado (cliente aprova)
pendente → rejeitado (cliente rejeita)
aprovado → pago (após upload do comprovante)
```

Regras

1. Apenas técnicos autenticados podem criar solicitações de material
2. A solicitação deve estar vinculada a uma visita (`visitId`) ou chamado (`ticketId`)
3. O cliente recebe a solicitação e pode aprovar ou rejeitar
4. Após aprovação, o cliente deve realizar o pagamento e fazer upload do comprovante
5. O comprovante é armazenado no S3 e a URL é salva em `receiptUrl`
6. O admin pode visualizar o comprovante na aba Materiais do detalhe do cliente

6.5 Geração do PDF de Relatório de Preventiva

O PDF é gerado inteiramente no **frontend** (browser) usando a biblioteca `jspdf` .

Estrutura do PDF

1. **Cabeçalho** (fundo vermelho CJR):
 - Logo CJR (embutida em base64 de `client/src/lib/cjrLogoB64.ts`)
 - Título “RELATÓRIO DE MANUTENÇÃO PREVENTIVA”
 - Data e hora da visita
2. **Dados da Visita:**
 - Nome do cliente e loja
 - Nome do técnico
 - Data agendada e hora
3. **Checklist** (36 itens em 3 colunas):
 - Ícone ✓ (verde) ou ✗ (vermelho) por item
 - Agrupado por categoria (Civil, Hidráulica, Elétrica)
4. **Observações do Técnico**
5. **Evidências Fotográficas:**

- Grade 2×N com as fotos capturadas durante o checklist
- Cada foto com legenda do item correspondente

6. Assinatura Digital:

- Imagem da assinatura capturada via `signature_pad`
- Nome do assinante

7. Rodapé (em todas as páginas):

- Número da página / total de páginas
- “Costa Júnior — Manutenção Preventiva”

Processo de Envio

1. PDF gerado como `Uint8Array` via `jspdf`
 2. Convertido para base64
 3. Enviado ao servidor via `techExecuteVisit` com campos `reportPdfB64` e `reportPdfMime`
 4. Servidor salva no S3 via `storagePut()`
 5. URL pública salva em `maintenanceVisits.reportPdfUrl`
-

6.6 Autenticação e Sessões

Sessões de Cliente e Técnico

- Token: 48 bytes aleatórios gerados com `crypto.randomBytes(48).toString("hex")`
- Armazenamento: tabelas `clientSessions` e `technicianSessions`
- Duração: 30 dias
- Renovação: não automática (usuário deve fazer login novamente após expiração)
- Invalidação: logout remove o registro da tabela

Verificação de Sessão

A verificação é feita manualmente em cada procedure que requer autenticação de cliente/técnico:

```
const token = req.cookies?.["cjr_client_session"];
if (!token) return null; // ou throw UNAUTHORIZED

const session = await db.select().from(clientSessions)
  .where(and(eq(clientSessions.token, token), gt(clientSessions.expiresAt, now)))
  .limit(1);

if (!session[0]) return null;
```

6.7 Códigos de Cliente e Loja

Geração de Código de Cliente

O código é gerado sequencialmente no formato `CLT-XXXX` onde `XXXX` é um número com 4 dígitos com zeros à esquerda.

Exemplo: `CLT-0001` , `CLT-0042` , `CLT-0100`

Geração de Código de Loja

O código da loja é derivado do código do cliente com sufixo `-LN` onde `N` é o número sequencial da loja.

Exemplo: `CLT-0001-L1` , `CLT-0001-L2` , `CLT-0001-L3`

6.8 Aprovação de Novas Lojas

Quando um cliente adiciona uma nova loja ao seu grupo (mesmo `clientId`), a loja é criada com `storeApprovalStatus = "pendente_aprovacao"`. O admin precisa aprovar manualmente antes que a loja fique ativa no sistema.

6.9 Controle de Acesso por Perfil (Usuários Internos)

Funcionalidade	Admin	Coordenador	Administrativo	Financeiro	Operacional	Comercial
Ver dados financeiros	✓	—	—	✓	—	✓
Criar/editar clientes	✓	—	—	—	—	—
Aprovar materiais	✓	—	—	—	—	—
Gerenciar técnicos	✓	—	—	—	—	—
Ver listagem de clientes	✓	✓	✓	✓	✓	✓
Ver agenda de visitas	✓	✓	✓	✓	✓	—
Gerenciar orçamentos	✓	—	—	—	—	—
Configurar preços	✓	—	—	—	—	—
Gerenciar cupons	✓	—	—	—	—	—
Editar contrato	✓	—	—	—	—	—

6.10 Cálculo Financeiro do Cliente

A procedure `adminGetClientDetail` calcula o resumo financeiro do cliente:

```
financial: {  
  totalMaintenance: sum(monthlyValue * months_active), // Estimativa de receita de m.  
  totalMaterials: sum(materialRequests.estimatedValue where status = "aprovado"),  
  totalGeneral: totalMaintenance + totalMaterials  
}
```

6.11 Cupons de Desconto

Regras de Aplicação

1. O cupom deve estar ativo (`isActive = true`)
2. A data atual deve estar entre `validFrom` e `validUntil` (se definidos)
3. `usedCount` deve ser menor que `maxUses` (se definido)
4. O plano do cliente deve estar em `applicablePlans` (se definido)
5. O desconto é aplicado sobre o valor mensal por `discountMonths` meses

Exemplo: Cupom de 15% por 3 meses em plano anual de R\$ 336/mês:

- Meses com desconto: $3 \times R336,00 \times 0,85 = R\ 856,80$
- Meses sem desconto: $9 \times R336,00 = R\ 3.024,00$
- Total: R3.880,80 (vs. R 4.032,00 sem cupom)

Portal CJR – Documentação Completa

Documento 07: Estrutura de Usuários e Permissões

7.1 Perfis de Acesso

O sistema possui **dois sistemas de identidade independentes**:

Sistema 1: Usuários Internos (Colaboradores CJR)

Autenticados via Manus OAuth. Armazenados na tabela `users`.

Perfil (<code>role</code>)	Descrição	Acesso Principal
<code>admin</code>	Administrador total do sistema	Todos os módulos, todas as ações
<code>coordenador</code>	Coordenador de operações	Módulos operacionais, sem financeiro
<code>administrativo</code>	Equipe administrativa	Documentos, fórum, treinamentos
<code>financeiro</code>	Equipe financeira	Dados financeiros, relatórios
<code>operacional</code>	Equipe operacional	Chamados, agenda, visitas
<code>comercial</code>	Equipe comercial	CRM de prospecção, dados comerciais

Sistema 2: Usuários Externos (Clientes e Técnicos)

Autenticados via e-mail + senha própria. Armazenados em `maintenanceClients` e `maintenanceTechnicians`.

Perfil	Tabela	Cookie de Sessão	Portal
Cliente (Lojista)	<code>maintenanceClients</code>	<code>cjr_client_session</code>	<code>/cliente</code>
Técnico	<code>maintenanceTechnicians</code>	<code>cjr_tech_session</code>	<code>/tecnico</code>

7.2 Permissões Detalhadas por Módulo

Módulo de Manutenção — Pannel Admin (/admin/manutencao)

Ação	Admin	Financeiro	Comercial	Outros
Ver aba Dashboard	✓	✓	✓	—
Ver aba Clientes	✓	✓	✓	—
Criar/editar cliente	✓	—	—	—
Ver dados financeiros do cliente	✓	✓	✓	—
Ver aba Leads	✓	—	✓	—
Converter lead em cliente	✓	—	—	—
Ver aba Técnicos	✓	—	—	—
Criar/editar técnico	✓	—	—	—
Vincular técnico a cliente	✓	—	—	—
Ver aba Preços	✓	—	—	—
Editar tabela de preços	✓	—	—	—
Ver aba Cupons	✓	—	—	—
Criar/editar cupons	✓	—	—	—
Ver aba Contrato	✓	—	—	—
Editar texto do contrato	✓	—	—	—
Ver aba Orçamentos	✓	—	—	—
Criar/responder orçamentos	✓	—	—	—
Ver aba Suporte	✓	—	—	—
Responder chamados de suporte	✓	—	—	—
Gerar link de reset de senha	✓	—	—	—
Gerar cronograma automático	✓	—	—	—
Aprovar/reprovar materiais	✓	—	—	—
Aprovar novas lojas	✓	—	—	—

Ação	Admin	Financeiro	Comercial	Outros
Exportar CSV	✓	✓	—	—

Módulo de Manutenção — Gestão (/manutencao/*)

Ação	Admin	Coordenador	Administrativo	Financeiro	Operacional	Comercial
Dashboard	✓	✓	✓	✓	✓	✓
Listar clientes	✓	✓	✓	✓	✓	✓
Ver detalhe do cliente	✓	✓	✓	✓	✓	✓
Chamados	✓	✓	✓	✓	✓	—
Agenda de visitas	✓	✓	✓	✓	✓	—
Técnicos	✓	—	—	—	—	—
Preventivos	✓	—	—	—	—	—
Orçamentos	✓	—	—	—	—	—

Portal do Cliente (/cliente)

O cliente tem acesso apenas aos dados das suas próprias lojas (identificadas pelo `clientCode`).

Ação	Cliente
Ver dashboard	✓
Ver chamados das suas lojas	✓
Criar chamado	✓
Ver preventivas das suas lojas	✓
Ver PDF de relatório	✓
Aprovar/rejeitar material	✓
Enviar comprovante de pagamento	✓
Ver orçamentos	✓
Aprovar/rejeitar/sugerir ajuste em orçamento	✓
Ver contrato	✓
Editar perfil (e-mail, telefone)	✓
Ver dados de outros clientes	✗
Criar visitas	✗
Criar orçamentos	✗

Portal do Técnico (/tecnico)

O técnico tem acesso **apenas às lojas às quais está vinculado** via `technicianClientLinks` .

Ação	Técnico
Ver lojas vinculadas	✓
Ver agenda de visitas	✓
Executar visita (checklist + assinatura + PDF)	✓
Ver chamados atribuídos	✓
Executar chamado	✓
Solicitar material	✓
Ver status das solicitações de material	✓
Abrir chamado de suporte	✓
Ver lojas de outros técnicos	✗
Aprovar materiais	✗
Criar orçamentos	✗
Ver dados financeiros	✗

7.3 Verificação de Permissões no Backend

Usuários Internos (OAuth)

```
// Verificação de role admin
if (ctx.user.role !== "admin") {
  throw new TRPCError({ code: "FORBIDDEN" });
}

// Verificação de acesso financeiro
const hasFinancialAccess = ["admin", "financeiro", "comercial"].includes(ctx.user.role);
```

Clientes (Cookie de Sessão)

```
// Verificação de sessão de cliente
const token = req.cookies?.["cjr_client_session"];
if (!token) return null;

const session = await db.select().from(clientSessions)
  .where(and(eq(clientSessions.token, token), gt(clientSessions.expiresAt, now)))
  .limit(1);

if (!session[0]) throw new TRPCError({ code: "UNAUTHORIZED" });

// Verificação de propriedade (cliente só vê suas lojas)
const client = await db.select().from(maintenanceClients)
  .where(eq(maintenanceClients.id, session[0].clientId))
  .limit(1);

// Busca todas as lojas do mesmo clientCode
const allStores = await db.select().from(maintenanceClients)
  .where(eq(maintenanceClients.clientCode, client[0].clientCode));

const storeIds = allStores.map(s => s.id);
// Filtra dados apenas para essas lojas
```

Técnicos (Cookie de Sessão)

```
// Verificação de sessão de técnico
const token = req.cookies?.["cjr_tech_session"];
// ... similar ao cliente

// Verificação de vínculo com cliente
const links = await db.select().from(technicianClientLinks)
  .where(eq(technicianClientLinks.technicianId, technicianId));

const linkedClientIds = links.map(l => l.clientId);
// Técnico só pode ver/executar visitas de clientes vinculados
```

7.4 Fluxo de Criação de Acesso

Para Novo Cliente

1. Admin cria o cliente em `/admin/manutencao` (aba Clientes)

2. Sistema gera `clientId` automaticamente (ex.: `CLT-0042`)
3. Admin clica em “Gerar Link de Senha” → sistema cria token de reset (24h)
4. Admin envia o link por e-mail ou WhatsApp para o cliente
5. Cliente acessa `/cliente/definir-senha?token=XXX` e define sua senha
6. Cliente pode fazer login em `/login-cliente`

Para Novo Técnico

1. Admin cria o técnico em `/admin/manutencao` (aba Técnicos)
2. Senha padrão: `tecnico1234` (com `mustChangePassword = true`)
3. Admin vincula o técnico às lojas que ele atenderá
4. Técnico faz login em `/login-tecnico` com e-mail e senha padrão
5. Sistema redireciona para `/tecnico/trocar-senha` na primeira sessão

Para Novo Colaborador Interno

1. Colaborador acessa o portal e faz login via Manus OAuth
2. Conta criada com `approvalStatus = "pending"` e `role = "operacional"` (padrão)
3. Admin acessa `/admin/users` e aprova o colaborador
4. Admin pode alterar o `role` conforme a função do colaborador

7.5 Restrições de Segurança

1. **Isolamento de dados:** Clientes não podem acessar dados de outros clientes. A verificação é feita via `clientId` em todas as queries do portal do cliente.
 2. **Isolamento de técnicos:** Técnicos só veem lojas vinculadas via `technicianClientLinks` . Não há acesso a dados de clientes não vinculados.
 3. **Senhas:** Armazenadas com bcrypt (salt rounds: 12). Nunca armazenadas em texto plano.
 4. **Tokens de reset:** Gerados com `crypto.randomBytes(32)` , válidos por 24 horas, invalidados após uso.
 5. **Sessões:** Tokens de 96 caracteres hex (48 bytes), armazenados no banco com expiração. Invalidados no logout.
 6. **Acesso admin:** Verificado explicitamente em cada procedure sensível via `ctx.user.role !== "admin"` .
 7. **Arquivos S3:** Chaves geradas com sufixo aleatório para evitar enumeração. Bucket público (URLs diretas sem assinatura).
-

Portal CJR – Documentação Completa

Documento 08: Manual de Implantação

8.1 Pré-requisitos

Requisito	Versão Mínima	Observação
Node.js	22.x	Recomendado: 22.13.0
pnpm	9.x	Gerenciador de pacotes
MySQL / TiDB	8.x / compatível	Banco de dados principal
AWS S3	—	Armazenamento de arquivos
Conta Manus	—	Para OAuth e APIs built-in
Conta Mercado Pago	—	Para pagamentos online

8.2 Instalação Local (Desenvolvimento)

Passo 1: Clonar o Repositório

```
git clone <URL_DO_REPOSITÓRIO> forum-cjr
cd forum-cjr
```

Passo 2: Instalar Dependências

```
pnpm install
```

Passo 3: Configurar Variáveis de Ambiente

Crie o arquivo `.env` na raiz do projeto com base no template abaixo:

```
# Banco de Dados
DATABASE_URL=mysql://usuario:senha@host:3306/forum_cjr

# Autenticação JWT
JWT_SECRET=sua_chave_secreta_muito_longa_e_aleatoria

# Manus OAuth (fornecido pela plataforma Manus)
VITE_APP_ID=seu_app_id_manus
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://accounts.manus.im
OWNER_OPEN_ID=seu_open_id
OWNER_NAME=Nome do Proprietário

# Manus Built-in APIs (fornecido pela plataforma Manus)
BUILT_IN_FORGE_API_URL=https://api.manus.im
BUILT_IN_FORGE_API_KEY=sua_chave_api_manus
VITE_FRONTEND_FORGE_API_KEY=sua_chave_api_frontend
VITE_FRONTEND_FORGE_API_URL=https://api.manus.im

# Analytics (opcional)
VITE_ANALYTICS_ENDPOINT=https://analytics.manus.im
VITE_ANALYTICS_WEBSITE_ID=seu_website_id

# Mercado Pago
MERCADO_PAGO_ACCESS_TOKEN=seu_access_token
MERCADO_PAGO_CLIENT_ID=seu_client_id
MERCADO_PAGO_CLIENT_SECRET=seu_client_secret
MERCADO_PAGO_PUBLIC_KEY=sua_public_key

# App
VITE_APP_TITLE=Portal CJR
VITE_APP_LOGO=/logo.png
VITE_APP_ID=forum-cjr
```

Nota: Em ambiente Manus, as variáveis são injetadas automaticamente pela plataforma. O arquivo `.env` é necessário apenas para desenvolvimento local.

Passo 4: Criar as Tabelas do Banco

```
pnpm db:push
```

Este comando executa `drizzle-kit generate` + `drizzle-kit migrate` para criar/atualizar todas as tabelas.

Passo 5: Iniciar o Servidor de Desenvolvimento

```
pnpm dev
```

O servidor estará disponível em `http://localhost:3000`.

8.3 Scripts Disponíveis

Script	Comando	Descrição
Desenvolvimento	<code>pnpm dev</code>	Inicia servidor com hot reload (tsx watch)
Build	<code>pnpm build</code>	Compila frontend (Vite) + backend (esbuild)
Testes	<code>pnpm test</code>	Executa testes Vitest
DB Push	<code>pnpm db:push</code>	Aplica alterações do schema ao banco
DB Studio	<code>pnpm db:studio</code>	Abre Drizzle Studio (UI do banco)

8.4 Estrutura de Configuração

`drizzle.config.ts` — Configuração do Drizzle

```
export default {
  schema: "./drizzle/schema.ts",
  out: "./drizzle/migrations",
  dialect: "mysql",
  dbCredentials: {
    url: process.env.DATABASE_URL!,
  },
};
```

`vite.config.ts` — Configuração do Vite

O Vite é configurado com:

- Plugin React para JSX
- Plugin Manus Runtime (injeção de variáveis de ambiente)
- Proxy de `/api` para o servidor Express em desenvolvimento

`server/_core/env.ts` — Variáveis de Ambiente no Servidor

Todas as variáveis de ambiente do servidor são acessadas via este arquivo. **Não acesse** `process.env` **diretamente** no código de aplicação — use sempre `env.ts`.

8.5 Deploy em Produção (Plataforma Manus)

O projeto é hospedado na plataforma Manus com deploy automático.

Processo de Deploy

1. Faça as alterações no código
2. Salve um checkpoint: `webdev_save_checkpoint`
3. Clique no botão **Publish** na interface da plataforma Manus
4. O sistema fará o build e deploy automaticamente

Domínios Configurados

- `portalcjr.vip` (domínio principal)
 - `www.portalcjr.vip`
 - `cjrforum-a6rpmddf.manus.space` (domínio Manus padrão)
-

8.6 Configuração do Banco de Dados

Conexão

O banco usa MySQL/TiDB. A string de conexão segue o formato:

```
mysql://USUARIO:SENHA@HOST:PORTA/NOME_DO_BANCO
```

Migrações

O Drizzle ORM gerencia as migrações. Para aplicar alterações no schema:

```
# Gera arquivos de migração
pnpm drizzle-kit generate

# Aplica as migrações
pnpm drizzle-kit migrate

# Atalho que faz os dois
pnpm db:push
```

Atenção: O comando `db:push` aplica as alterações diretamente. Em produção, sempre faça backup antes de executar migrações.

8.7 Configuração do S3

O armazenamento de arquivos usa AWS S3 via helper `server/storage.ts`.

Configuração

As credenciais S3 são injetadas automaticamente pela plataforma Manus via `BUILT_IN_FORGE_API_KEY` e `BUILT_IN_FORGE_API_URL`.

Uso no Código

```
import { storagePut } from "../storage";

// Upload de arquivo
const fileKey = `clients/${clientId}/receipts/${Date.now()}-receipt.pdf`;
const { url } = await storagePut(fileKey, fileBuffer, "application/pdf");

// url é a URL pública do arquivo no S3
```

8.8 Configuração do Mercado Pago

Variáveis Necessárias

```
MERCADO_PAGO_ACCESS_TOKEN=APP_USR- ...
MERCADO_PAGO_CLIENT_ID= ...
MERCADO_PAGO_CLIENT_SECRET= ...
MERCADO_PAGO_PUBLIC_KEY=APP_USR- ...
```


Obtenção das Credenciais

1. Acesse mercadopago.com.br/developers
2. Crie uma aplicação
3. Copie as credenciais de produção (ou sandbox para testes)

8.9 Configuração do Manus OAuth

O Manus OAuth é usado para autenticação dos colaboradores internos da CJR.

Variáveis Necessárias

```
VITE_APP_ID=forum-cjr
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://accounts.manus.im
```

Fluxo OAuth

1. Frontend redireciona para `VITE_OAUTH_PORTAL_URL/login?app_id=VITE_APP_ID&state= ...`
2. Usuário faz login no portal Manus
3. Manus redireciona para `/api/oauth/callback?code= ...&state= ...`
4. Servidor troca o code por token e cria sessão
5. Usuário é redirecionado para a URL original

8.10 Primeiro Acesso ao Sistema

Configurar o Primeiro Administrador

1. Acesse o portal e faça login via Manus OAuth
2. No banco de dados, atualize o `role` do usuário para `"admin"`:

```
UPDATE users SET role = 'admin', approvalStatus = 'approved'
WHERE email = 'seu@email.com';
```

1. Acesse `/admin/manutencao` para começar a configurar o sistema

Configurações Iniciais Recomendadas

1. **Tabela de preços:** Acesse a aba “Preços” e configure os valores base por tipo de loja

2. **Texto do contrato:** Acesse a aba “Contrato” e insira o texto do contrato de adesão
 3. **Primeiro cliente:** Acesse a aba “Clientes” e cadastre o primeiro cliente
 4. **Primeiro técnico:** Acesse a aba “Técnicos” e cadastre o primeiro técnico
-

8.11 Monitoramento e Logs

Logs do Servidor

Em desenvolvimento, os logs são exibidos no terminal. Em produção na plataforma Manus, os logs estão disponíveis em:

- `.manus-logs/devserver.log` — Logs do servidor
- `.manus-logs/browserConsole.log` — Logs do browser
- `.manus-logs/networkRequests.log` — Requisições HTTP

Verificação de Saúde

Para verificar se o servidor está rodando:

```
curl http://localhost:3000/api/trpc/auth.me
```

Resposta esperada: `{"result":{"data":{"json":null}}}` (usuário não autenticado)

Portal CJR — Documentação Completa

Documento 09: Manual do Usuário

9.1 Manual do Cliente (Lojista)

Primeiro Acesso

1. Você receberá um link por e-mail ou WhatsApp da equipe CJR no formato:
`https://portalcjr.vip/cliente/definir-senha?token=XXXXXXXX`
2. Acesse o link e defina sua senha (mínimo 6 caracteres).
3. Após definir a senha, acesse o portal em: `https://portalcjr.vip/login-cliente`

4. Insira seu e-mail e a senha que você acabou de definir.

Navegando pelo Portal

Após o login, você verá o menu lateral com as seguintes seções:

Dashboard: Visão geral com gráfico de chamados por tipo no último ano e contadores de chamados abertos e preventivas realizadas.

Lojas: Aqui você gerencia todas as suas lojas. Se você tiver mais de uma loja no mesmo grupo, todas aparecerão aqui.

Plano: Informações sobre o seu plano de manutenção (tipo, valor, serviços incluídos, vigência).

Termo: Acesso ao contrato de adesão assinado.

Abrindo um Chamado de Manutenção

1. No menu lateral, clique em **Lojas**
 2. Selecione a loja onde ocorreu o problema
 3. Clique na aba **Chamados**
 4. Clique no botão **+ Novo Chamado**
 5. Preencha os campos:
 - **Título:** Resumo do problema (ex.: "Torneira com vazamento")
 - **Descrição:** Detalhes do problema
 - **Categoria:** Elétrica, Hidráulica, Civil, Ar-condicionado, Equipamentos ou Outro
 - **Prioridade:** Baixa, Média, Alta ou Urgente
 - **Localização:** Onde na loja está o problema (ex.: "Banheiro feminino")
 - **Foto:** Tire uma foto do problema (opcional, mas recomendado)
 6. Clique em **Enviar**
-

Aprovando uma Solicitação de Material

Quando o técnico precisar de materiais para realizar o serviço, você receberá uma solicitação:

1. Acesse **Lojas** → selecione a loja → aba **Materiais**
 2. Você verá a solicitação com descrição e valor estimado
 3. Clique em **✓ Aprovar** para autorizar a compra, ou **✗ Rejeitar** para recusar
 4. Se aprovado, realize o pagamento pelo link disponível
 5. Após o pagamento, clique em  **Enviar Comprovante**
 6. Selecione a imagem ou PDF do comprovante e clique em **Confirmar envio**
-

Respondendo a um Orçamento

Quando a CJR enviar um orçamento para um serviço extra:

1. Acesse **Lojas** → selecione a loja → aba **Orçamentos**
2. Clique em **Ver PDF** para visualizar o orçamento detalhado
3. Você tem três opções:
 - **✓ Aprovar:** Aceita o orçamento como está
 - **✗ Rejeitar:** Recusa o orçamento (informe o motivo)
 - **💬 Sugerir Ajuste:** Envie uma sugestão de modificação para a CJR revisar

Se você sugerir um ajuste, a CJR analisará sua sugestão e poderá enviar um novo orçamento revisado.

Acompanhando Visitas Preventivas

1. Acesse **Lojas** → selecione a loja → aba **Preventivas**
 2. Você verá o histórico de todas as visitas preventivas
 3. Para visitas concluídas, clique em  **Ver Relatório** para baixar o PDF do relatório
 4. O card no topo mostra a data da próxima preventiva agendada
-

Editando seu Perfil

1. Clique no ícone de perfil no canto superior direito
 2. Você pode atualizar seu e-mail e telefone
 3. Clique em **Salvar** para confirmar as alterações
-

9.2 Manual do Técnico

Primeiro Acesso

1. Acesse `https://portalcjr.vip/login-tecnico`
 2. Use o e-mail cadastrado pelo administrador e a senha padrão: `tecnico1234`
 3. No primeiro login, você será redirecionado para trocar a senha
 4. Defina uma nova senha segura
-

Visualizando suas Lojas

Na aba **Lojas**, você verá todas as lojas que estão vinculadas a você. Para cada loja, você pode acessar:

- **Chamados:** Chamados atribuídos a você nessa loja

- **Preventivas:** Histórico de visitas preventivas
 - **Checklist:** Checklist da última preventiva
 - **Materiais:** Solicitações de material
-

Verificando a Agenda

Na aba **Agenda**, você verá todas as visitas agendadas para você. O badge colorido indica a urgência da próxima preventiva:

-  Verde: mais de 7 dias
-  Amarelo: 3 a 7 dias
-  Vermelho: menos de 3 dias ou atrasada

Clique no badge para ver todas as lojas com preventiva no mesmo dia.

Executando uma Visita Preventiva

Passo 1: Iniciar a visita

1. Acesse a aba **Agenda** ou **Lojas** → **Preventivas**
2. Localize a visita agendada e clique em **Iniciar Visita**

Passo 2: Preencher o Checklist O checklist possui 36 itens divididos em três grupos:

- **CIVIL (12 itens):** Pisos, paredes, portas, impermeabilização, pintura, etc.
- **HIDRÁULICA (12 itens):** Torneiras, vasos, caixas d'água, bombas, etc.
- **ELÉTRICA (12 itens):** Quadro elétrico, disjuntores, tomadas, iluminação, etc.

Para cada item:

1. Marque o checkbox se o item foi verificado e está OK
2. Clique no ícone de câmera  para tirar uma foto como evidência
3. A foto é obrigatória para itens marcados como concluídos

Passo 3: Observações Preencha o campo de observações com qualquer informação relevante sobre a visita.

Passo 4: Assinatura Digital

1. Solicite ao responsável da loja que assine na tela
2. O responsável desenha a assinatura com o dedo ou mouse
3. Preencha o nome do assinante
4. Clique em **Confirmar Assinatura**

Passo 5: Gerar Relatório

1. Clique em **Gerar Relatório e Finalizar**
 2. O sistema gerará automaticamente um PDF com:
 - Logo CJR e dados da visita
 - Checklist completo com status de cada item
 - Fotos das evidências
 - Assinatura digital
 3. O PDF é enviado ao servidor e fica disponível para o cliente
-

Executando um Chamado Corretivo

1. Acesse a aba **Chamados**
 2. Localize o chamado atribuído a você
 3. Clique em **Executar**
 4. Atualize o status conforme o andamento:
 - **Em Andamento** — quando iniciar o serviço
 - **Aguardando Material** — se precisar de materiais
 - **Concluído** — quando finalizar
 5. Ao concluir, é obrigatório tirar uma foto do serviço realizado
 6. Preencha a descrição da resolução
-

Solicitando Material

Se você precisar de materiais durante uma visita ou chamado:

1. Clique em **Solicitar Material** (disponível na execução de chamados e visitas)
 2. Preencha:
 - **Descrição:** O que precisa (ex.: “Registro de gaveta 1/2 polegada”)
 - **Valor Estimado:** Custo aproximado do material
 3. Clique em **Enviar Solicitação**
 4. O cliente receberá a solicitação e poderá aprovar ou rejeitar
 5. Você pode acompanhar o status na aba **Materiais** da loja
-

Abrindo um Chamado de Suporte

Se você tiver algum problema técnico ou precisar de suporte da equipe CJR:

1. Acesse a aba **Suporte** (ícone de headset no menu)
2. Clique em **+ Novo Chamado de Suporte**

3. Descreva o problema detalhadamente
 4. Anexe uma foto se necessário
 5. Clique em **Enviar**
 6. A equipe CJR responderá em breve
-

9.3 Manual do Administrador CJR

Acessando o Painel Administrativo

1. Acesse `https://portalcjr.vip` e faça login com sua conta Manus
 2. No menu principal, acesse **Admin** → **Manutenção**
 3. Você verá o painel administrativo completo em `/admin/manutencao`
-

Cadastrando um Novo Cliente

1. Acesse a aba **Clientes**
2. Clique em **+ Novo Cliente**
3. Preencha todos os dados:
 - Dados da empresa (razão social, CNPJ, responsável)
 - Dados da loja (nome, shopping, endereço, tipo, área)
 - Serviços incluídos (Elétrica, Hidráulica, Civil)
 - Tipo de plano (Trimestral, Semestral, Anual)
4. O sistema calculará automaticamente o valor mensal e total
5. Clique em **Salvar**
6. O sistema gerará um código único para o cliente (ex.: `CLT-0042`)

Após o cadastro:

1. Clique em **Gerar Link de Senha** para o cliente
 2. Envie o link por e-mail ou WhatsApp
 3. Quando o cliente ativar a conta, gere o cronograma automático de preventivas
-

Gerando o Cronograma Automático de Preventivas

1. Acesse `/admin/manutencao/clientes/:id` (detalhe do cliente)
2. Clique na aba **Preventivas**
3. Clique em **Cronograma Automático**
4. Selecione a data da primeira visita

5. Revise as datas geradas automaticamente (editáveis antes de confirmar)
6. Clique em **Confirmar e Gerar**

O sistema criará todas as visitas do plano:

- Plano trimestral: 3 visitas
- Plano semestral: 6 visitas
- Plano anual: 12 visitas

Enviando um Orçamento para o Cliente

1. Acesse a aba **Orçamentos** no painel admin
2. Clique em **+ Novo Orçamento**
3. Selecione o cliente
4. Preencha título, descrição e valor
5. Faça upload do PDF do orçamento
6. Clique em **Enviar**
7. O cliente receberá o orçamento no portal e poderá responder

Se o cliente sugerir um ajuste:

1. Você verá a sugestão do cliente na lista de orçamentos
2. Clique em **Responder**
3. Faça upload do novo PDF revisado
4. Clique em **Reenviar**
5. O cliente receberá o novo orçamento para avaliação

Cadastrando um Novo Técnico

1. Acesse a aba **Técnicos**
2. Clique em **+ Novo Técnico**
3. Preencha nome, e-mail, telefone, CPF e especialidades
4. Clique em **Salvar**
5. O técnico receberá acesso com a senha padrão `tecnico1234`
6. Vincule o técnico às lojas que ele atenderá clicando em **Vincular a Cliente**

Gerenciando Chamados de Suporte

1. Acesse a aba **Suporte**

2. Você verá todos os chamados abertos pelos técnicos
 3. Clique em **Responder** para adicionar notas e alterar o status
 4. Status disponíveis: Aberto, Em Andamento, Resolvido, Fechado
-

Portal CJR — Documentação Completa

Documento 10: Código Fonte Organizado

10.1 Estrutura de Pastas Detalhada

forum-cjr/	
├── client/	# Frontend React (SPA)
│ ├── index.html	# Entry point HTML com Google Fonts
│ ├── public/	# Assets estáticos (copiados para /)
│ │ └── logo.png	# Logo CJR
│ └── src/	
│ ├── App.tsx	# ★ Roteamento principal (Wouter)
│ │	# Todos os <Route> do sistema
│ ├── main.tsx	# Providers: ThemeProvider, trRPC, QueryClient
│ ├── index.css	# Variáveis CSS globais (Tailwind 4 tokens)
│ ├── const.ts	# getLoginUrl(), constantes globais
│ └── components/	
│ ├── ui/	# Componentes shadcn/ui (NÃO editar)
│ │ ├── button.tsx	
│ │ ├── dialog.tsx	
│ │ ├── tabs.tsx	
│ │ └── ... (30+ componentes)	
│ ├── DashboardLayout.tsx	# Layout sidebar para painéis internos
│ ├── ManutencaoLayout.tsx	# ★ Sidebar do módulo de manutenção
│ ├── ComercialLayout.tsx	# Sidebar do módulo comercial
│ ├── AIChatBox.tsx	# Componente de chat com IA
│ ├── ErrorBoundary.tsx	# Tratamento de erros React
│ └── Map.tsx	# Integração Google Maps
│ └── contexts/	
│ └── ThemeContext.tsx	# Contexto de tema (light/dark)
│ └── hooks/	
│ ├── useMobile.tsx	# Detecção de dispositivo móvel
│ └── usePersistFn.ts	# Hook para funções persistentes
│ └── lib/	
│ ├── trpc.ts	# ★ Configuração do cliente trRPC
│ ├── utils.ts	# cn() e outros utilitários
│ └── cjrLogoB64.ts	# ★ Logo CJR em base64 (para PDF)
│ └── pages/	
│ ├── Home.tsx	# Landing page / hub de navegação
│ ├── Chat.tsx	# Fórum de dúvidas com IA
│ ├── Dashboard.tsx	# Dashboard do usuário interno
│ ├── Treinamentos.tsx	# Biblioteca de treinamentos
│ ├── GestaoTecnica.tsx	# Gestão técnica (estoque, equipamentos)
│ ├── DocumentosTecnicos.tsx	# Documentos técnicos
│ ├── TermoEntrega.tsx	# Geração de Termo de Entrega
│ └── AtaReuniao.tsx	# Geração de Ata de Reunião

—	Perfil.tsx	# Perfil do usuário
—	MinhasSubmissoes.tsx	# Submissões de conhecimento
—	Onboarding.tsx	# Onboarding de novos colaboradores
—	TecnicoPortal.tsx	# ★ Portal do técnico (/tecnico)
—	ClientePortal.tsx	# ★ Portal do cliente (/cliente)
—	ClienteLogin.tsx	# Login do cliente
—	TecnicoLogin.tsx	# Login do técnico
—	ClienteDefinirSenha.tsx	# Definir senha (token)
—	TecnicoDefinirSenha.tsx	# Definir senha (token)
—	ClienteTrocarSenha.tsx	# Trocar senha (autenticado)
—	TecnicoTrocarSenha.tsx	# Trocar senha (autenticado)
—	admin/	
—	AdminManutencao.tsx	# ★ Painel admin de manutenção
—	AdminClienteDetalhe.tsx	# ★ Detalhe do cliente (admin)
—	AdminDashboard.tsx	# Dashboard admin geral
—	Users.tsx	# Gestão de usuários
—	Documents.tsx	# Gestão de documentos
—	Questions.tsx	# Gestão de perguntas
—	Knowledge.tsx	# Base de conhecimento
—	TechnicalDocuments.tsx	# Documentos técnicos (admin)
—	AdminConteudo.tsx	# Gestão de conteúdo
—	AdminTrainingReport.tsx	# Relatório de treinamentos
—	MapaPaginas.tsx	# Mapa de páginas
—	AdminSubmissoesConhecimento.tsx	
—	manutencao/	
—	DashboardManutencao.tsx	# Dashboard gestão
—	ClientesManutencao.tsx	# Listagem de clientes
—	ClienteDetalhe.tsx	# Detalhe do cliente (gestão)
—	ChamadosManutencao.tsx	# Gestão de chamados
—	AgendaManutencao.tsx	# Agenda de visitas
—	TecnicosManutencao.tsx	# Gestão de técnicos
—	PreventivosManutencao.tsx	# Gestão de preventivos
—	OrcamentosManutencao.tsx	# ★ Gestão de orçamentos
—	ContratarManutencao.tsx	# Contratação pública
—	comercial/	
—	DashboardComercial.tsx	# Dashboard comercial
—	Prospeccao.tsx	# CRM de prospecção
—	PadroesComerciais.tsx	# Padrões comerciais
—	FerramentasComerciais.tsx	# Ferramentas comerciais
—	server/	# Backend Node.js/Express
—	—_core/	# Framework (NÃO editar)
—	—index.ts	# Entry point: Express + tRPC + OAuth
—	—trpc.ts	# publicProcedure, protectedProcedure
—	—context.ts	# Contexto: { user, req, res }
—	—oauth.ts	# Manus OAuth handler
—	—cookies.ts	# Configuração de cookies (SameSite, etc.)

llm.ts	# invokeLLM() helper
notification.ts	# notifyOwner() helper
env.ts	# Variáveis de ambiente tipadas
imageGeneration.ts	# generateImage() helper
voiceTranscription.ts	# transcribeAudio() helper
— routers/	
— manutencao.ts	# ★ Router admin de manutenção
— clientAuth.ts	# ★ Router cliente/técnico + portais
— _clientDetail.ts	# Helper: adminGetClientDetail
— auth.ts	# Router OAuth (me, logout)
— documents.ts	# Router de documentos
— knowledge.ts	# Router da base de conhecimento
— chat.ts	# Router do chat com IA
— users.ts	# Router de usuários
— training.ts	# Router de treinamentos
— inventory.ts	# Router de estoque
— prospect.ts	# Router de prospecção comercial
— ...	
— db.ts	# ★ Helpers de query (getDb, etc.)
— storage.ts	# ★ storagePut(), storageGet()
— routers.ts	# Agregador de todos os routers
— drizzle/	
— schema.ts	# ★ Definição de TODAS as tabelas
— migrations/	# Arquivos SQL de migração
— shared/	
— types.ts	# Tipos compartilhados
— package.json	# Dependências e scripts
— tsconfig.json	# TypeScript (strict mode)
— vite.config.ts	# Vite: React plugin + proxy /api
— drizzle.config.ts	# Drizzle: schema path + dialect
— todo.md	# Lista de tarefas do projeto

10.2 Padrões de Código

Convenções de Nomenclatura

Tipo	Padrão	Exemplo
Componentes React	PascalCase	ClientePortal.tsx
Funções/variáveis	camelCase	generateToken()
Constantes	UPPER_SNAKE_CASE	SESSION_COOKIE_CLIENT
Tabelas do banco	camelCase	maintenanceClients
Campos do banco	camelCase	contractStatus
Procedures tRPC	camelCase	clientLogin, techExecuteVisit
Rotas URL	kebab-case	/login-cliente, /admin/manutencao

Padrão de Procedure tRPC

```
// ✅ Padrão correto para procedure protegida
procedureName: protectedProcedure
.input(z.object({
  id: z.number(),
  name: z.string().min(1).max(255),
  optional: z.string().optional(),
}))
.mutation(async ({ input, ctx }) => {
  // Verificação de permissão
  if (ctx.user.role !== "admin") {
    throw new TRPCError({ code: "FORBIDDEN" });
  }

  const db = await getDb();
  if (!db) throw new Error("Banco de dados indisponível.");

  // Lógica de negócio
  await db.insert(table).values({ ...input });

  return { ok: true };
}),
```

Padrão de Verificação de Sessão de Cliente

```
// ✅ Padrão para procedures do portal do cliente
clientProcedure: publicProcedure
  .input(z.object({ storeId: z.number() }))
  .query(async ({ input, ctx }) => {
    const req = (ctx as any).req as Request;
    const token = req.cookies?.[SESSION_COOKIE_CLIENT];
    if (!token) throw new TRPCError({ code: "UNAUTHORIZED" });

    const db = await getDb();
    if (!db) throw new Error("Banco de dados indisponível.");

    const now = new Date();
    const sessions = await db.select().from(clientSessions)
      .where(and(
        eq(clientSessions.token, token),
        gt(clientSessions.expiresAt, now)
      ))
      .limit(1);

    if (!sessions[0]) throw new TRPCError({ code: "UNAUTHORIZED" });

    const clientId = sessions[0].clientId;
    // ... lógica de negócio
  }),
```

Padrão de Upload para S3

```
// ✅ Padrão para upload de arquivo
if (input.fileBase64 && input.fileMime) {
  const buffer = Buffer.from(input.fileBase64, "base64");
  const ext = input.fileMime.includes("pdf") ? "pdf" : "png";
  const randomSuffix = Math.random().toString(36).substring(2, 10);
  const fileKey = `quotes/${quoteId}/quote-${randomSuffix}.${ext}`;
  const { url } = await storagePut(fileKey, buffer, input.fileMime);
  fileUrl = url;
}
```

Padrão de Query no Frontend

```
// ✅ Query com estado de loading e erro
const { data: clients, isLoading, error } = trpc.manutencao.listClients.useQuery(
  { search: searchTerm },
  { enabled: !!searchTerm } // Só executa se searchTerm não for vazio
);

if (isLoading) return <Spinner />;
if (error) return <ErrorMessage message={error.message} />;
if (!clients?.length) return <EmptyState />;
```

Padrão de Mutation no Frontend

```
// ✅ Mutation com invalidação de cache
const utils = trpc.useUtils();

const createVisit = trpc.manutencao.createVisit.useMutation({
  onSuccess: () => {
    // Invalida as queries relacionadas para refetch automático
    utils.manutencao.listClients.invalidate();
    setModalOpen(false);
    toast.success("Visita agendada com sucesso!");
  },
  onError: (err) => {
    toast.error(err.message);
  },
});

// Chamada da mutation
createVisit.mutate({
  clientId: selectedClient.id,
  scheduledDate: "2026-07-15",
  scheduledTime: "09:00",
});
```

10.3 Arquivos Críticos e Suas Funções

drizzle/schema.ts

Define todas as tabelas do banco de dados. **Toda alteração no banco começa aqui.** Após editar, execute `pnpm db:push`.

`server/routers/manutencao.ts`

Router principal do módulo de manutenção. Contém todas as procedures acessíveis por usuários internos (admin, coordenador, etc.). Inclui a lógica de precificação e geração de cronograma.

`server/routers/clientAuth.ts`

Router de autenticação e portais de clientes e técnicos. Contém o sistema de sessão independente (cookie-based) e todas as procedures dos portais `/cliente` e `/tecnico`.

`server/routers/_clientDetail.ts`

Helper que consolida todos os dados de um cliente em uma única query. Usado pela tela de detalhe do cliente no admin.

`client/src/pages/TecnicoPortal.tsx`

Portal completo do técnico. Contém o checklist de 36 itens, a lógica de captura de fotos, assinatura digital e geração do PDF com jsPDF.

`client/src/pages/ClientePortal.tsx`

Portal completo do cliente. Contém os fluxos de aprovação de materiais, negociação de orçamentos e upload de comprovantes.

`client/src/pages/admin/AdminManutencao.tsx`

Painel administrativo principal. Contém todas as abas de gestão (clientes, técnicos, leads, preços, cupons, contrato, orçamentos, suporte).

`client/src/pages/admin/AdminClienteDetalhe.tsx`

Página de detalhe do cliente com 6 abas. Contém o modal de cronograma automático com preview das datas.

`client/src/lib/cjrLogoB64.ts`

Logo CJR em formato base64 para embutir no PDF de preventiva. Se o logo mudar, este arquivo deve ser atualizado.

10.4 Notas sobre TypeScript

O projeto usa TypeScript em modo strict. Há aproximadamente 48 erros de tipo no arquivo

`server/routers/manutencao.ts` relacionados a importações de tipos do Drizzle ORM que **não afetam o funcionamento** em runtime. Estes erros existem desde o início do projeto e podem ser ignorados nas verificações de CI.

Para verificar erros TypeScript reais:


```
pnpm tsc --noEmit 2>&1 | grep -v "manutencao.ts"
```

10.5 Testes

O projeto usa Vitest para testes unitários. O arquivo de referência é `server/auth.logout.test.ts` .
Para executar os testes:

```
pnpm test
```

Para executar com cobertura:

```
pnpm test --coverage
```

Áreas que precisam de cobertura de testes:

- Lógica de precificação (`calculatePrice`)
- Geração de cronograma automático
- Verificação de sessão de cliente/técnico
- Fluxo de negociação de orçamentos

Portal CJR — Documentação Completa

Documento 11: Roadmap Futuro

11.1 Bugs Conhecidos (Pendentes de Correção)

Os seguintes bugs foram identificados no checkpoint `8317a97c` e ainda não foram corrigidos:

#	Bug	Componente	Impacto
1	Erro <code>toISOString</code> no cronograma automático de preventivas	<code>server/routers/manutencao.ts</code>	Médio — cronograma pode falhar em alguns casos
2	Botão de envio de orçamento sumiu no admin	<code>AdminManutencao.tsx</code>	Alto — admin não consegue enviar orçamentos pela aba
3	Comprovante de material voltando para status “pendente” após upload	<code>clientAuth.ts</code>	Alto — cliente precisa reenviar comprovante
4	Aba Financeiro mostrando R\$ 0,00	<code>_clientDetail.ts</code>	Médio — cálculo financeiro incorreto

11.2 Melhorias de Curto Prazo (1-3 meses)

Notificações em Tempo Real

Implementar notificações push para alertar clientes e técnicos sobre:

- Nova solicitação de material pendente
- Orçamento enviado pelo admin
- Resposta do cliente ao orçamento
- Visita agendada/reagendada

Tecnologia sugerida: WebSockets via `socket.io` ou Server-Sent Events

Envio Automático de E-mail

Configurar envio automático de e-mails para:

- Boas-vindas ao novo cliente (com link de criação de senha)
- Lembrete de preventiva (3 dias antes)
- Confirmação de aprovação de material
- Notificação de novo orçamento

Tecnologia: Nodemailer (já instalado) ou Resend (já instalado)

Relatório de Preventiva Melhorado

- Incluir dados da loja (endereço, shopping) no cabeçalho do PDF

- Adicionar QR Code no rodapé para verificação de autenticidade
 - Opção de envio automático do PDF por e-mail ao cliente após conclusão
-

Dashboard Administrativo Aprimorado

- Gráfico de receita mensal (últimos 12 meses)
 - Mapa de calor de chamados por região/shopping
 - Indicadores de SLA (tempo médio de resolução de chamados)
 - Alertas de clientes inadimplentes
-

App Mobile (PWA)

Transformar o portal do técnico em Progressive Web App (PWA) para:

- Funcionamento offline (checklist sem internet)
 - Câmera nativa do dispositivo para fotos
 - Geolocalização automática do check-in/check-out
 - Notificações push nativas
-

11.3 Melhorias de Médio Prazo (3-6 meses)

Módulo de Contratos Digitais

- Geração automática de contrato em PDF ao ativar cliente
 - Assinatura digital do contrato via DocuSign ou similar
 - Histórico de versões do contrato
-

Integração com WhatsApp Business

- Envio automático de mensagens via WhatsApp para:
 - Confirmação de agendamento
 - Lembrete de preventiva
 - Status de chamados
 - Chatbot para abertura de chamados via WhatsApp
-

Módulo de Faturamento

- Geração automática de boletos/NFS-e
- Integração com sistemas de contabilidade (Omie, ContaAzul)

- Controle de inadimplência com alertas automáticos
 - Histórico de pagamentos por cliente
-

Gestão de Estoque de Materiais

- Controle de estoque de materiais frequentemente usados
 - Alertas de estoque mínimo
 - Integração com solicitações de material dos técnicos
 - Relatório de consumo por cliente/loja
-

Portal de Relatórios Gerenciais

- Relatório de produtividade por técnico
 - Relatório de recorrência de problemas por tipo
 - Análise de custo por cliente
 - Exportação para Excel/PDF
-

11.4 Melhorias de Longo Prazo (6-12 meses)

Inteligência Artificial para Manutenção Preditiva

- Análise de padrões nos checklists para prever falhas
 - Sugestão automática de frequência de visitas por tipo de loja
 - Classificação automática de chamados por urgência via NLP
-

Marketplace de Materiais

- Catálogo de materiais com preços de fornecedores parceiros
 - Cotação automática ao criar solicitação de material
 - Compra direta pelo portal com entrega na loja
-

Multi-empresa

- Suporte a múltiplas empresas de manutenção na mesma plataforma
 - Isolamento completo de dados por empresa
 - Painel de super-admin para gestão de todas as empresas
-

Integração com IoT

- Sensores de temperatura e umidade nas lojas
- Alertas automáticos de anomalias
- Histórico de dados ambientais nos relatórios de preventiva

11.5 Débitos Técnicos

Item	Prioridade	Descrição
Testes unitários	Alta	Cobertura de testes para lógica de negócio crítica
Tratamento de erros	Alta	Melhorar mensagens de erro para o usuário final
Paginação	Média	Adicionar paginação nas listagens (clientes, visitas, chamados)
Cache	Média	Implementar cache de queries frequentes (Redis)
Logs estruturados	Média	Implementar logging estruturado (Winston/Pino)
Auditoria	Baixa	Expandir <code>actionLogs</code> para cobrir mais ações do sistema
Internacionalização	Baixa	Suporte a múltiplos idiomas (i18n)
Acessibilidade	Baixa	Melhorar conformidade com WCAG 2.1
Performance	Baixa	Otimizar queries N+1 no Drizzle ORM
Documentação de código	Baixa	Adicionar JSDoc nas funções principais

11.6 Sugestões de Arquitetura Futura

Separação de Serviços

À medida que o sistema crescer, considerar separar em microserviços:

- **Serviço de Autenticação:** OAuth + sessões de cliente/técnico
- **Serviço de Manutenção:** Lógica de negócio principal
- **Serviço de Notificações:** E-mail, WhatsApp, push
- **Serviço de Relatórios:** Geração de PDFs e relatórios

Migração para PostgreSQL

O TiDB/MySQL tem limitações para queries analíticas complexas. Considerar migração para PostgreSQL com extensão TimescaleDB para dados de séries temporais (histórico de visitas, métricas).

CDN para Assets

Configurar CDN (CloudFront) para servir os PDFs e imagens armazenados no S3, melhorando a performance de carregamento dos relatórios.
